

HAMPARO – PROYECTO FINAL DIN

Eduardo Moreno Rodríguez

Índice

CAPÍTULO 1 – MARCO GENERAL DEL PROYECTO	9
1.1 Introducción y presentación	9
1.2 Contexto social actual	10
1.3 Necesidad social detectada	10
1.4 Justificación de la solución propuesta.....	10
1.5 Objetivos del proyecto	11
Objetivo general	11
Objetivos específicos.....	11
CAPÍTULO 2 – RA1: DESARROLLO DE INTERFACES	11
2.1 RA1.a – Análisis de herramientas y librerías utilizadas.....	11
2.1.1 Selección del entorno de desarrollo	11
2.1.2 Elección de Jetpack Compose frente a XML.....	12
2.1.3 Librerías adicionales empleadas	12
2.2 RA1.b – Creación de la interfaz gráfica.....	12
2.2.1 Proceso de diseño previo	12
2.2.2 Pantallas implementadas	13
2.2.3 Coherencia visual entre pantallas	13
2.3 RA1.c – Uso de layouts y posicionamiento.....	14
2.3.1 Análisis de necesidades de layout.....	14
2.3.2 Uso de Column	14
2.3.3 Uso de Row	15
2.3.4 Uso de Box	16
2.3.5 Uso de LazyColumn	16
2.3.6 Justificación global del posicionamiento	17
2.4 RA1.d – Personalización de componentes	17
2.4.1 Identidad visual	17
2.4.2 Colores	18
2.4.3 Tipografía	18
2.4.4 Componentes personalizados	19
2.4.5 Modales y feedback al usuario	19
2.5 RA1.e – Análisis del código de la interfaz	20

2.5.1 Organización del código	20
2.5.2 Legibilidad y mantenimiento	21
2.5.3 Patrones utilizados en la interfaz	21
2.5.4 Ejemplo de componente reutilizable	21
2.5.5 Gestión del estado en la interfaz	22
2.6 RA1.f – Modificación y mejora del código.....	23
2.6.1 Refactorización de componentes	23
2.6.2 Mejora progresiva del diseño visual	23
2.6.3 Optimización del uso de layouts.....	24
2.6.4 Adaptación a pruebas de uso.....	24
2.6.5 Valoración final de las modificaciones.....	24
2.7 RA1.g – Asociación de eventos e interacción.....	25
2.7.1 Gestión de eventos de usuario	25
2.7.2 Uso de callbacks y eventos en Jetpack Compose	25
2.7.3 Fluidez y coherencia de la interacción	26
2.7.4 Prevención de errores y confirmación de acciones	26
2.8 RA1.h – Integración global de la aplicación.....	26
2.8.1 Integración de pantallas y navegación	27
2.8.2 Integración de componentes reutilizables.....	27
2.8.3 Integración del estado y la lógica de interacción	27
2.8.4 Estabilidad y comportamiento global	27
CAPÍTULO 3 – RA2: NUEVAS FORMAS DE INTERACCIÓN	28
Nota previa sobre el alcance del RA2.....	28
3.1 RA2.a – Herramientas de Nuevas Interfaces de Usuario (NUI)	28
3.1.1 Concepto de Nuevas Interfaces de Usuario	28
3.1.2 Herramientas NUI disponibles en el ecosistema Android	29
3.1.3 Justificación del uso de NUI en Hamparo	29
3.2 RA2.b – Diseño conceptual de la interacción NUI.....	30
3.2.1 Enfoque de diseño centrado en el usuario.....	30
3.2.2 Integración progresiva de las NUI	31
3.2.3 Escenarios de uso definidos	31
3.2.4 Diseño de flujos de interacción alternativos	31

3.2.5 Accesibilidad como eje del diseño	32
3.2.6 Coherencia con el diseño general de la aplicación	32
3.3 RA2.c – Interacción por voz	32
3.3.1 Justificación del uso de la voz como interfaz	33
3.3.2 Herramientas de reconocimiento de voz en Android	33
3.3.3 Casos de uso propuestos para la interacción por voz	33
3.3.4 Diseño del flujo de interacción por voz.....	34
3.3.5 Feedback al usuario	34
3.3.6 Limitaciones y riesgos de la interacción por voz.....	34
3.3.7 Seguridad y control del usuario	35
3.4 RA2.d – Interacción por gestos.....	35
3.4.1 Concepto de interacción por gestos	35
3.4.2 Justificación del uso de gestos en Hamparo	36
3.4.3 Gestos propuestos y su funcionalidad	36
3.4.4 Diseño de la experiencia de usuario basada en gestos.....	36
3.4.5 Accesibilidad y gestos	37
3.4.6 Prevención de errores mediante gestos	37
3.4.7 Integración con el resto de la interfaz.....	37
3.5 RA2.e – Detección facial y corporal	38
3.5.1 Concepto y posibilidades	38
3.5.2 Aplicación potencial en Hamparo	38
3.5.3 Beneficios y riesgos.....	39
3.5.4 Aspectos éticos y de privacidad	39
3.5.5 Integración con el resto de la aplicación	40
3.6 RA2.f – Realidad aumentada	40
3.6.1 Concepto de realidad aumentada y su evolución	40
3.6.2 Herramientas de realidad aumentada en Android.....	41
3.6.3 Aplicación conceptual de la realidad aumentada en Hamparo	41
3.6.4 Ventajas de la realidad aumentada en aplicaciones asistenciales	41
3.6.5 Limitaciones técnicas y de usabilidad	42
3.6.6 Consideraciones de accesibilidad y seguridad	42
3.7 Comparativa entre interfaz tradicional y NUI.....	42

CAPÍTULO 4 – RA3: COMPONENTES REUTILIZABLES	43
4.1 RA3.a – Herramientas para la creación de componentes	43
4.1.1 Componentes en Jetpack Compose	43
4.1.2 Ventajas frente a enfoques tradicionales	43
4.1.3 Organización del código por componentes	44
4.2 RA3.b – Diseño y creación de componentes reutilizables	44
4.2.1 Identificación de elementos reutilizables	44
4.2.2 Ejemplos de componentes creados	44
4.2.3 Beneficios obtenidos.....	45
4.3 RA3.c – Parámetros y valores por defecto	45
4.3.1 Parametrización de componentes	45
4.3.2 Uso de valores por defecto	45
4.4 RA3.d – Gestión de eventos en componentes	46
4.4.1 Asociación de eventos mediante callbacks	46
4.4.2 Fluidez de la interacción	46
4.5 RA3.f – Documentación de componentes.....	46
4.5.1 Importancia de la documentación.....	46
4.5.2 Documentación aplicada en el proyecto	46
4.6 RA3.h – Integración de componentes en la aplicación	47
4.6.1 Uso en múltiples pantallas	47
4.6.2 Impacto en la calidad del proyecto	47
CAPÍTULO 5 – RA4: USABILIDAD Y DISEÑO VISUAL	47
5.1 RA4.a – Aplicación de estándares de diseño	47
5.1.1 Estándares de diseño en Android	47
5.1.2 Aplicación práctica de los estándares en Hamparo	48
5.1.3 Adaptación de los estándares al contexto del proyecto.....	48
5.2 RA4.b – Valoración y justificación de los estándares	48
5.2.1 Análisis crítico de los estándares utilizados	48
5.2.2 Decisiones de diseño justificadas	49
5.3 RA4.c – Diseño y uso de menús	49
5.3.1 Tipos de menús utilizados	49
5.3.2 Integración de los menús en la navegación	49

5.4 RA4.d – Distribución de acciones.....	49
5.4.1 Priorización de acciones principales.....	49
5.4.2 Prevención de errores en la distribución	49
5.5 RA4.e – Distribución de controles	50
5.5.1 Organización y jerarquía visual.....	50
5.5.2 Espaciado y alineación	50
5.6 RA4.f – Elección de controles	50
5.6.1 Selección adecuada de componentes.....	50
5.6.2 Justificación de la elección	50
5.7 RA4.g – Diseño visual global	50
5.7.1 Estética y legibilidad.....	50
5.7.2 Coherencia visual	50
5.8 RA4.h – Claridad de mensajes	51
5.8.1 Lenguaje utilizado	51
5.8.2 Adaptación al usuario.....	51
5.9 RA4.i – Pruebas de usabilidad	51
5.9.1 Metodología de pruebas	51
5.9.2 Resultados obtenidos.....	51
5.10 RA4.j – Evaluación en distintos dispositivos	51
5.10.1 Dispositivos utilizados.....	51
5.10.2 Resultados de la evaluación	51
CAPÍTULO 6 – RA5: INFORMES	51
Nota previa sobre el alcance de los informes (RA5)	52
6.1 RA5.a – Estructura del informe	52
6.1.1 Concepto de informe en aplicaciones móviles	52
6.1.2 Estructura general de los informes en Hamparo	52
6.1.3 Justificación de la estructura propuesta.....	53
6.1.4 Coherencia con el diseño de la aplicación	53
6.2 RA5.b – Generación de informes a partir de fuentes de datos.....	53
6.2.1 Identificación de las fuentes de datos.....	53
6.2.2 Tratamiento de los datos para la generación de informes	54
6.3 RA5.c – Filtros sobre los valores a presentar	54

6.3.1 Importancia del uso de filtros en informes	54
6.3.2 Tipos de filtros propuestos en Hamparo.....	55
6.3.3 Integración de los filtros en la interfaz.....	55
6.4 RA5.d – Valores calculados, recuentos y totales	55
6.4.1 Tipos de valores calculados incluidos.....	55
6.4.2 Utilidad de los valores calculados para el usuario	56
6.4.3 Integración de los cálculos en los informes.....	56
6.5 RA5.e – Gráficos generados a partir de los datos	56
6.5.1 Importancia de la representación gráfica	57
6.5.2 Tipos de gráficos utilizados o propuestos.....	57
6.5.3 Generación de gráficos a partir de datos reales	57
6.5.4 Integración de los gráficos en los informes.....	58
6.5.5 Usabilidad y claridad de los gráficos.....	58
6.6 RA5.f – Uso de herramientas para la generación de informes (FFOE)	58
6.7 RA5.g – Modificación del código del informe (FFOE)	59
6.8 RA5.h – Integración de informes en la aplicación (FFOE).....	59
CAPÍTULO 7 – RA6: DESPLIEGUE Y DISTRIBUCIÓN DE LA APLICACIÓN	60
7.1 RA6.a – Preparación del entorno de despliegue	60
7.2 RA6.b – Generación del paquete de la aplicación.....	60
7.3 RA6.c – Instalación en dispositivos reales	60
7.4 RA6.d – Gestión de versiones y actualizaciones	61
7.5 RA6.e – Consideraciones de seguridad en el despliegue.....	61
7.6 RA6.f – Valoración final del despliegue	61
CAPÍTULO 8 – RA7: DISTRIBUCIÓN DE LA APLICACIÓN.....	61
Nota previa sobre el alcance de la distribución (RA7)	62
8.1 RA7.a – Empaquetado de la aplicación	62
8.1.1 Ciclo de empaquetado en Android	62
8.1.2 Formatos de empaquetado: APK y AAB.....	62
8.1.3 Elección del formato de empaquetado	62
8.2 RA7.b – Personalización del instalador.....	63
8.2.1 Elementos personalizables del instalador.....	63
8.2.2 Importancia de la personalización.....	63

8.2.3 Buenas prácticas aplicadas	63
8.3 RA7.c – Generación del paquete desde el entorno de desarrollo	64
8.3.1 Proceso detallado en Android Studio	64
8.3.2 Control de versiones de la aplicación	64
8.3.3 Verificación del paquete generado	65
8.4 RA7.d – Uso de herramientas externas	65
8.4.1 Tipos de herramientas externas utilizadas o consideradas	65
8.4.2 Beneficios del uso de herramientas externas	65
8.4.3 Aplicación al proyecto Hamparo	66
8.5 RA7.e – Firma digital de la aplicación	66
8.5.1 Concepto de firma digital en Android	66
8.5.2 Proceso de firma de la aplicación	66
8.5.3 Importancia de la gestión del keystore	67
8.5.4 Documentación del proceso de firma	67
8.6 RA7.f – Instalación desatendida.....	67
8.6.1 Contextos de uso de la instalación desatendida.....	67
8.6.2 Aplicación teórica en el proyecto Hamparo	68
8.6.3 Ventajas de la instalación desatendida	68
8.7 RA7.g – Proceso de desinstalación	68
8.7.1 Funcionamiento del proceso de desinstalación en Android.....	68
8.7.2 Gestión de datos durante la desinstalación.....	69
8.7.3 Buenas prácticas relacionadas con la desinstalación.....	69
8.8 RA7.h – Canales y estrategia de distribución.....	69
8.8.1 Canales de distribución disponibles.....	69
8.8.2 Estrategia de distribución propuesta	70
8.8.3 Adaptación al tipo de aplicación	70
CAPÍTULO 9 – RA8: PRUEBAS Y VALIDACIÓN.....	70
9.1 RA8.a – Estrategia de pruebas	71
9.1 RA8.a – Estrategia de pruebas	71
9.1.1 Objetivos de la estrategia de pruebas	71
9.1.2 Enfoque general adoptado	71
9.1.3 Alcance de las pruebas	72

9.2 RA8.b – Pruebas de integración	72
9.2.1 Alcance de las pruebas de integración.....	72
9.2.2 Metodología aplicada	73
9.2.3 Resultados obtenidos.....	73
9.3 RA8.c – Pruebas de regresión	73
9.3.1 Justificación de las pruebas de regresión	74
9.3.2 Aplicación práctica en el proyecto Hamparo	74
9.3.3 Resultados de las pruebas de regresión	74
9.4 RA8.d – Pruebas de volumen y estrés	74
9.4.1 Objetivo de las pruebas de volumen	75
9.4.2 Objetivo de las pruebas de estrés.....	75
9.4.3 Aplicación teórica en el proyecto Hamparo	75
9.4.4 Resultados esperados y medidas preventivas	76
9.5 RA8.e – Pruebas de seguridad	76
9.5.1 Objetivos de las pruebas de seguridad	76
9.5.2 Aspectos evaluados	76
9.5.3 Aplicación práctica en Hamparo	77
9.6 RA8.f – Uso de recursos	77
9.6.1 Recursos analizados	77
9.6.2 Metodología aplicada	77
9.6.3 Resultados obtenidos.....	78
9.7 RA8.g – Documentación de las pruebas	78
9.7.1 Importancia de la documentación de pruebas	78
9.7.2 Contenido de la documentación de pruebas.....	78
9.7.3 Relación con el proceso de desarrollo	79
CAPÍTULO 10 – CONCLUSIONES FINALES	79
10.1 Conclusión general.....	79
10.2 Valoración personal del proyecto.....	79
10.3 Posibles mejoras futuras	80
ANEXOS	80
A. Evidencias gráficas	80
B. Repositorio del proyecto.....	80

CAPÍTULO 1 – MARCO GENERAL DEL PROYECTO

1.1 Introducción y presentación



Figura 1. Pantalla de inicio de la aplicación Hamparo.

Pantalla de bienvenida que actúa como punto de entrada principal para el usuario. Se aprecia una jerarquía visual clara, una identidad gráfica coherente y un botón de acción principal (“Comenzar”) que guía de forma intuitiva al siguiente paso de la aplicación. El diseño limpio y centrado refuerza la sensación de seguridad y accesibilidad desde el primer contacto.

Nombre de la aplicación: Hamparo

Plataforma: Android

Tecnología principal: Jetpack Compose (Kotlin)

Modalidad: Proyecto individual

Hamparo es una aplicación móvil Android desarrollada como proyecto final del módulo Desarrollo de Interfaces. Su objetivo principal es ofrecer una solución digital orientada al cuidado, la supervisión y la asistencia de personas dependientes o en situación de vulnerabilidad, facilitando la gestión y el acceso a información relevante por parte de distintos perfiles de usuario.

Desde el primer contacto, la aplicación busca transmitir confianza, claridad y sencillez, aspectos fundamentales en aplicaciones relacionadas con la salud y el bienestar. La

pantalla inicial actúa como una pantalla de entrada o *landing screen*, permitiendo comenzar el uso de la aplicación de forma inmediata y sin sobrecargar al usuario con información innecesaria.

1.2 Contexto social actual

El uso de aplicaciones móviles es hoy en día imprescindible en la vida cotidiana. Sin embargo, muchas aplicaciones presentan problemas de usabilidad, exceso de información y falta de accesibilidad. Estas carencias afectan especialmente a:

- Personas mayores.
- Personas con diversidad funcional.
- Usuarios con bajo nivel de alfabetización digital.
- Personas en situaciones de emergencia o estrés.

En estos contextos, una mala interfaz puede provocar errores graves o impedir el uso correcto de la aplicación. Hamparo surge como respuesta a esta necesidad, ofreciendo una aplicación que centraliza la información y facilita la interacción entre cuidadores, familiares y pacientes, mejorando la calidad del cuidado y la tranquilidad de todas las partes implicadas.

1.3 Necesidad social detectada

Durante el análisis previo al desarrollo se detectaron las siguientes necesidades:

- Diseñar una interfaz clara, accesible y coherente.
- Aplicaciones accesibles sin curvas de aprendizaje complejas.
- Integración de ayudas visuales, sonoras y contextuales.
- Herramientas que prioricen la seguridad del usuario.
- Crear una aplicación escalable y mantenible.

Hamparo se plantea como una herramienta de apoyo digital que reduzca la carga cognitiva del usuario y facilite la toma de decisiones rápidas.

1.4 Justificación de la solución propuesta

La solución propuesta se justifica por:

- La necesidad real detectada en distintos colectivos.
- La ausencia de aplicaciones simples y accesibles orientadas a la ayuda.
- El potencial de Jetpack Compose para crear interfaces modernas.
- La aplicabilidad directa de los conocimientos del módulo.

1.5 Objetivos del proyecto

Objetivo general

Desarrollar una aplicación Android funcional y accesible mediante Jetpack Compose, aplicando criterios profesionales de diseño de interfaces.

Objetivos específicos

- Diseñar una interfaz clara y coherente.
- Implementar varias pantallas bien estructuradas.
- Diferenciar tipos de usuario.
- Crear componentes reutilizables.
- Aplicar criterios de accesibilidad y usabilidad.
- Documentar todo el proceso de desarrollo.

CAPÍTULO 2 – RA1: DESARROLLO DE INTERFACES

El presente capítulo desarrolla en profundidad el **Resultado de Aprendizaje RA1**, abordando de forma detallada todos sus criterios (RA1.a a RA1.h). Se analizan las decisiones técnicas y de diseño adoptadas durante el desarrollo de la aplicación Hamparo, justificando cada una de ellas desde un punto de vista profesional, de usabilidad y de accesibilidad.

2.1 RA1.a – Análisis de herramientas y librerías utilizadas

2.1.1 Selección del entorno de desarrollo

Para el desarrollo del proyecto se ha utilizado **Android Studio** como entorno de desarrollo integrado (IDE). Esta elección se justifica por ser la herramienta oficial recomendada por Google para el desarrollo de aplicaciones Android, ofreciendo soporte completo para Kotlin, Jetpack Compose, emulación de dispositivos, análisis de rendimiento y depuración.

Android Studio permite además una integración directa con sistemas de control de versiones como Git, facilitando el trabajo ordenado y la trazabilidad de los cambios realizados durante el desarrollo del proyecto.

2.1.2 Elección de Jetpack Compose frente a XML

Jetpack Compose ha sido seleccionado como tecnología principal para la creación de la interfaz gráfica frente al sistema tradicional basado en XML. Esta decisión se fundamenta en los siguientes aspectos:

- Modelo **declarativo**, que facilita la comprensión del estado de la interfaz.
- Reducción significativa de código repetitivo.
- Mayor facilidad para crear componentes reutilizables.
- Mejor adaptación a distintos tamaños de pantalla.
- Integración nativa con Kotlin.

Desde un punto de vista profesional, Jetpack Compose representa el presente y futuro del desarrollo de interfaces Android, por lo que su uso resulta coherente con un entorno laboral real.

2.1.3 Librerías adicionales empleadas

Durante el desarrollo se han utilizado diversas librerías complementarias:

- **Material 3**: para garantizar una interfaz moderna, accesible y coherente con las guías de diseño de Android.
- **Navigation Compose**: para la gestión de la navegación entre pantallas.
- **APIs del sistema**: notificaciones, alarmas y biometría.

Cada librería ha sido seleccionada atendiendo a criterios de estabilidad, documentación oficial y compatibilidad.

2.2 RA1.b – Creación de la interfaz gráfica

2.2.1 Proceso de diseño previo

Antes de comenzar la implementación técnica, se realizó un diseño conceptual de la aplicación. Este proceso incluyó:

- Identificación de las pantallas necesarias.
- Definición de las acciones principales del usuario.
- Priorización de funcionalidades críticas.

Este enfoque permitió evitar improvisaciones durante la fase de desarrollo y asegurar una estructura coherente.

2.2.2 Pantallas implementadas

La aplicación Hamparo cuenta con varias pantallas bien diferenciadas:



Figura 2. Pantalla de selección de rol de usuario. Pantalla que permite identificar el tipo de usuario que accede a la aplicación, adaptando posteriormente la experiencia de uso y las funcionalidades disponibles según el perfil seleccionado.

- Pantalla de inicio: punto de entrada a la aplicación.
- Pantalla principal de usuario: acceso a funciones clave.
- Pantalla de administración: configuración avanzada.
- Pantallas de ayuda y tutorial.
- Pantallas de configuración.

Cada pantalla ha sido diseñada manteniendo coherencia visual y funcional, reutilizando componentes comunes.

2.2.3 Coherencia visual entre pantallas

Se ha mantenido una línea gráfica uniforme mediante:

- Uso consistente de colores.
- Tipografías homogéneas.
- Espaciados similares.
- Comportamientos de navegación predecibles.

Esto reduce la curva de aprendizaje y mejora la experiencia de usuario.

2.3 RA1.c – Uso de layouts y posicionamiento

2.3.1 Análisis de necesidades de layout

Cada pantalla de la aplicación presenta necesidades de organización distintas. Por este motivo, no se ha utilizado un único layout de forma indiscriminada, sino que se ha seleccionado el más adecuado en cada caso.

2.3.2 Uso de Column



Figura 3. Pantalla de edición de datos personales estructurada mediante un layout Column, organizando los campos de forma vertical para facilitar la lectura y la introducción de datos.

El layout **Column** se ha utilizado principalmente para:

- Organizar elementos verticalmente.
- Crear formularios simples.
- Disponer botones de acción principales.

Su uso favorece la lectura natural de arriba hacia abajo, especialmente adecuada para usuarios con dificultades tecnológicas.

2.3.3 Uso de Row

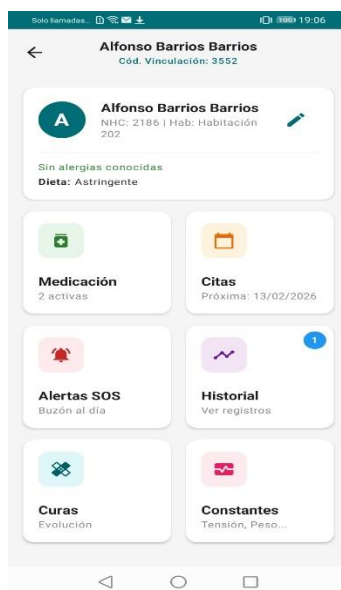


Figura 4. Uso del layout Row para organizar iconos y textos de forma horizontal en los accesos principales de la aplicación, mejorando la claridad visual y la rapidez de interacción.

El layout **Row** se ha empleado para:

- Agrupar iconos con texto.
- Distribuir acciones secundarias.
- Crear encabezados simples.

Se ha evitado la sobrecarga horizontal para no dificultar la lectura.

2.3.4 Uso de Box



Figura 5. Ejemplo de uso del layout Box para superponer un diálogo modal sobre la pantalla principal, permitiendo destacar información y centrar la atención del usuario.

El layout **Box** permite superponer elementos y se ha utilizado en:

- Pantallas con elementos destacados.
- Indicadores visuales.
- Componentes con fondo personalizado.

2.3.5 Uso de LazyColumn

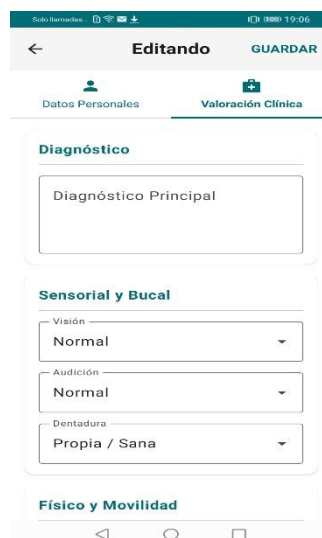


Figura 6. Pantalla de valoración clínica con contenido dinámico mostrado mediante LazyColumn, permitiendo el desplazamiento vertical eficiente y una carga optimizada de elementos.

Para la representación de listas dinámicas se ha utilizado **LazyColumn**, permitiendo:

- Optimización del rendimiento.
- Carga eficiente de elementos.
- Mejor experiencia en dispositivos con recursos limitados.

2.3.6 Justificación global del posicionamiento

La combinación de layouts ha permitido una jerarquía visual clara, evitando interfaces caóticas y facilitando la interacción rápida.

2.4 RA1.d – Personalización de componentes

2.4.1 Identidad visual



Figura 7. Identidad visual de la aplicación basada en colores suaves, tipografía legible y un enfoque cercano orientado a mejorar la experiencia del usuario.

Se ha definido una identidad visual propia para la aplicación, evitando el uso de estilos por defecto sin personalización, adaptándose a las personas mayores y teniendo en cuenta su accesibilidad.

2.4.2 Colores



Figura 8. Uso del color como elemento informativo para indicar estados críticos, mejorando la comprensión inmediata del estado del sistema.

Los colores utilizados han sido seleccionados teniendo en cuenta:

- Contraste suficiente para accesibilidad.
- Asociación emocional con seguridad y calma.
- Coherencia en toda la aplicación.

2.4.3 Tipografía

Se ha utilizado una tipografía legible, con tamaños adecuados para:

- Usuarios con dificultades visuales.
- Uso en exteriores.
- Situaciones de estrés.

Como puedes ver en la imagen anterior.

2.4.4 Componentes personalizados

Se han creado botones y tarjetas personalizadas, reutilizables en distintas pantallas, lo que mejora la coherencia y reduce duplicación de código.



Figura 9. Botón de acción crítica personalizado, diseñado para destacar visualmente y facilitar una respuesta rápida en situaciones de emergencia.

2.4.5 Modales y feedback al usuario

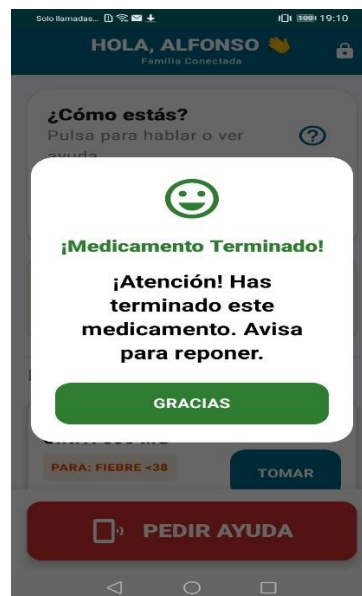


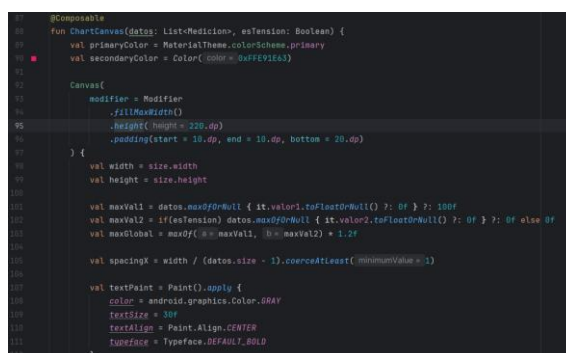
Figura 10. Ventana modal de confirmación que proporciona feedback visual inmediato al usuario tras completar una acción.

- Feedback inmediato
- Mensaje claro
- Color verde = acción correcta
- Botón único → no confunde

2.5 RA1.e – Análisis del código de la interfaz

El código de la interfaz de Hamparo se ha desarrollado siguiendo criterios de **claridad**, **modularidad** y **mantenibilidad**, evitando estructuras complejas innecesarias y priorizando siempre que otro desarrollador pueda entender y modificar el proyecto sin dificultad.

En lugar de limitarse a componentes estándar, se ha implementado un sistema propio de gráficos utilizando Canvas, lo que permite adaptar completamente la representación de los datos a las necesidades del usuario.



```

@Composable
fun ChartCanvas(datos: List<Medicion>, esFusion: Boolean) {
    val primaryColor = MaterialTheme.colorScheme.primary
    val secondaryColor = Color(0xFFE91E63)

    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(120.dp)
            .padding(start = 10.dp, end = 10.dp, bottom = 20.dp)
    ) {
        val width = size.width
        val height = size.height

        val maxVal1 = datos.mapNotNull { it.valor1.toFloatOrNull() }.ifEmpty { 100f }
        val maxVal2 = if(esFusion) datos.mapNotNull { it.valor2.toFloatOrNull() }.ifEmpty { 0f } else 0f
        val maxLabel = maxOf(0, maxVal1, 0, maxVal2) * 1.2f

        val spacingX = width / (datos.size - 1).coerceAtLeast(1)

        val textPaint = Paint().apply {
            color = android.graphics.Color.GRAY
            textSize = 10f
            textAlign = Paint.Align.CENTER
            typeface = Typeface.DEFAULT_BOLD
        }
    }
}

```

Figura 12. Implementación de una gráfica personalizada mediante Canvas. A partir de las mediciones almacenadas, se dibujan líneas, puntos y etiquetas de fecha, generando una gráfica visualmente clara y adaptada al contexto de la aplicación.

2.5.1 Organización del código

El código se ha estructurado siguiendo principios de modularidad:

- Separación por **pantallas** (por ejemplo, pantalla de login, pantalla principal de paciente, pantalla de medicación, etc.).
- Creación de **componentes reutilizables** para elementos visuales que se repiten (botones, tarjetas, cabeceras).
- Ficheros específicos para la **navegación** y rutas principales de la aplicación.
- Agrupación lógica de funciones relacionadas dentro del mismo archivo para facilitar su localización.

Esta organización evita tener archivos excesivamente largos y promueve una visión clara del flujo de la aplicación.

2.5.2 Legibilidad y mantenimiento

Se ha priorizado un código limpio y legible, teniendo en cuenta los siguientes aspectos:

- Uso de **nombres descriptivos** para funciones, parámetros y variables (por ejemplo, PatientCard, MedicationList, onAlertClick).
- Funciones composables **cortas y con una única responsabilidad**, evitando que una sola función gestione demasiados elementos a la vez.
- Uso puntual de **comentarios** para aclarar partes clave del código, especialmente en secciones donde se gestiona lógica de estado o navegación.
- Evitar anidaciones excesivas de composables, separando en funciones auxiliares cuando ha sido necesario.

Todo ello facilita el mantenimiento del proyecto y permite que futuras ampliaciones puedan realizarse sin reescribir grandes bloques de código.

2.5.3 Patrones utilizados en la interfaz

Aunque se trata de un proyecto de ámbito educativo, se han tenido en cuenta patrones utilizados en entornos profesionales, entre ellos:

- **Descomposición en capas visuales**: separación entre lo que se muestra en pantalla y la lógica que decide qué mostrar.
- **Lifting state up** (elevación del estado): en lugar de que cada componente gestione su propio estado de forma aislada, se ha optado por centralizar estados en niveles superiores cuando era necesario, y pasarlos como parámetros a los composables.
- Uso de **callbacks** (onClick, onValueChange, etc.) para comunicar acciones del usuario hacia niveles superiores de la lógica.

Estos patrones hacen que la interfaz sea más fácil de probar, depurar y extender.

2.5.4 Ejemplo de componente reutilizable

Un ejemplo representativo del análisis del código es la creación de un componente reutilizable para representar la información de un paciente o de un medicamento. Este componente recibe parámetros como el nombre, la descripción y las acciones asociadas, y se encarga únicamente de la representación visual.

```

28 fun HealthChart(
29     datos: List<Medicion>,
30     titulo: String,
31     unidad: String,
32     modifier: Modifier = Modifier
33 ) {
34     // 1. Convertimos los datos para asegurarnos de que se puedan pintar
35     val datosValidos = remember(key1 = datos) {
36         datos.filter { it.valor1.toFloatOrNull() != null }
37     }
38
39     // Detectamos si es Tensión (tiene valor2)
40     val esTension = remember(key1 = datosValidos) {
41         datosValidos.any { it.valor2.toFloatOrNull() != null }
42     }
43
44     Card(
45         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
46         shape = RoundedCornerShape(size = 16.dp),
47         colors = CardDefaults.cardColors(containerColor = Color.White),
48         modifier = modifier.fillMaxWidth()
49     ) {
50         Column(modifier = Modifier.padding(8.dp)) {
51             Row(verticalAlignment = Alignment.CenterVertically) {
52                 Text(text = "■", style = MaterialTheme.typography.headlineSmall)
53                 Spacer(modifier = Modifier.width(8.dp))
54                 Column {
55                     Text(
56                         text = titulo,
57                         style = MaterialTheme.typography.titleMedium,
58                         fontWeight = FontWeight.Bold,
59                         color = MaterialTheme.colorScheme.primary
60                     )
61                 }
62             }
63         }
64     }
65 }

```

Figura 12. Implementación de una gráfica personalizada mediante Canvas. A partir de las mediciones almacenadas, se dibujan líneas, puntos y etiquetas de fecha, generando una gráfica visualmente clara y adaptada al contexto de la aplicación.

Gracias a este enfoque:

- Cuando es necesario cambiar el diseño de la tarjeta, basta con modificar el componente en un único punto.
- Se reduce el riesgo de inconsistencias visuales entre pantallas.
- Se minimiza la duplicación de código.

2.5.5 Gestión del estado en la interfaz

El estado de la interfaz se ha gestionado siguiendo principios de claridad y previsibilidad:

- Se utilizan **variables de estado** (`remember { mutableStateOf(...) }` o estados proporcionados desde ViewModels) para controlar elementos como diálogos, mensajes de error o indicadores de carga.
- Los **eventos de usuario** (por ejemplo, pulsar un botón o seleccionar un elemento de una lista) actualizan el estado, y es el propio estado el que provoca la actualización visual de la UI.
- Se ha evitado modificar el mismo estado desde múltiples lugares sin control, lo que podría generar comportamientos inesperados.

Este enfoque está alineado con la filosofía declarativa de Jetpack Compose: *la interfaz es una función del estado*.

2.6 RA1.f – Modificación y mejora del código

Durante el desarrollo de la aplicación Hamparo, el código ha pasado por varias fases de revisión y mejora. Estas modificaciones no se han realizado de forma aleatoria, sino como resultado de pruebas de uso, detección de problemas de mantenibilidad y necesidad de mejorar la claridad del diseño de la interfaz.

Desde las primeras versiones del proyecto, se detectó la importancia de mantener una estructura clara y flexible que permitiera introducir cambios sin afectar al funcionamiento global de la aplicación.

Empezamos en una base de datos local, y acabos emigrando a firebase.

2.6.1 Refactorización de componentes

Una de las principales mejoras realizadas ha sido la **refactorización de componentes**. Inicialmente, algunas pantallas contenían demasiada lógica visual concentrada en una sola función composible. Esto dificultaba la lectura del código y complicaba su reutilización.

Para solucionar este problema, se procedió a:

- Dividir pantallas grandes en **componentes más pequeños**.
- Extraer elementos visuales repetidos a composables reutilizables.
- Separar la lógica de presentación del contenido dinámico.

Esta refactorización ha permitido reducir la duplicación de código y mejorar la consistencia visual entre pantallas.

2.6.2 Mejora progresiva del diseño visual

A lo largo del desarrollo se realizaron ajustes visuales orientados a mejorar la experiencia de usuario, tales como:

- Aumento del tamaño de botones críticos para facilitar su uso.
- Mejora del contraste de colores en estados de alerta o advertencia.
- Ajuste de márgenes y espaciados para evitar interfaces sobrecargadas.

Estas mejoras se implementaron tras comprobar que, en ciertas pantallas, el exceso de información o controles demasiado pequeños podía dificultar la interacción, especialmente en usuarios mayores o en situaciones de estrés.

2.6.3 Optimización del uso de layouts

Otra mejora importante fue la optimización del uso de layouts. En versiones iniciales se utilizaban estructuras más complejas de lo necesario, lo que hacía el código menos claro.

Posteriormente se revisó cada pantalla para:

- Simplificar la jerarquía de layouts.
- Sustituir combinaciones innecesarias por estructuras más claras.
- Mejorar el rendimiento visual en pantallas con listas o contenido dinámico.

Esto permitió una interfaz más limpia y un código más fácil de mantener.

2.6.4 Adaptación a pruebas de uso

Las modificaciones realizadas también responden a pruebas de uso realizadas durante el desarrollo. Estas pruebas permitieron detectar:

- Flujos de navegación poco claros.
- Acciones que requerían demasiados pasos.
- Elementos que no transmitían correctamente su función.

A partir de estas observaciones, se ajustaron textos, botones y comportamientos de la interfaz, mejorando la fluidez y reduciendo la posibilidad de error por parte del usuario.

2.6.5 Valoración final de las modificaciones

Las modificaciones y mejoras aplicadas al código de la interfaz han contribuido a obtener una aplicación más robusta, coherente y profesional. El proceso de revisión continua demuestra una actitud crítica hacia el propio trabajo y una aproximación similar a la que se sigue en entornos profesionales de desarrollo de software.

Gracias a estas mejoras, la aplicación Hamparo presenta un código más mantenible, una interfaz más clara y una experiencia de usuario mejor adaptada a su finalidad social.

2.7 RA1.g – Asociación de eventos e interacción

La correcta asociación de eventos y la gestión de la interacción con el usuario es un aspecto clave en el diseño de interfaces. En la aplicación Hamparo se ha puesto especial énfasis en que todas las acciones del usuario generen una respuesta clara, inmediata y coherente, evitando comportamientos inesperados o confusos.

La interacción ha sido diseñada teniendo en cuenta que la aplicación puede utilizarse en situaciones de estrés o urgencia, por lo que la simplicidad y la previsibilidad de las acciones son fundamentales.

2.7.1 Gestión de eventos de usuario

La aplicación responde correctamente a los principales eventos de interacción, entre los que destacan:

- Pulsaciones sobre botones de acción principales (por ejemplo, solicitar ayuda, registrar datos o confirmar acciones).
- Selección de elementos dentro de listas dinámicas.
- Cambios de estado provocados por formularios o controles interactivos.
- Confirmaciones y cancelaciones mediante diálogos y modales.

Cada evento está asociado a una acción concreta dentro de la aplicación, evitando ambigüedades y garantizando que el usuario comprenda qué efecto tiene cada interacción.

2.7.2 Uso de callbacks y eventos en Jetpack Compose

En la implementación de la interfaz se han utilizado los mecanismos propios de Jetpack Compose para la gestión de eventos, principalmente mediante **callbacks** (onClick, onChange, etc.).

Este enfoque permite:

- Separar claramente la lógica de interacción del diseño visual.
- Facilitar la reutilización de componentes.
- Centralizar la gestión de acciones importantes en niveles superiores del código.

Gracias a este sistema, los componentes visuales no ejecutan acciones de forma directa, sino que notifican al sistema cuando ocurre un evento, manteniendo el código desacoplado y más fácil de mantener.

2.7.3 Fluidez y coherencia de la interacción

Uno de los objetivos principales del diseño de la interacción ha sido lograr una experiencia fluida y coherente entre pantallas. Para ello se han aplicado los siguientes criterios:

- Respuesta inmediata a las acciones del usuario, evitando retardos innecesarios.
- Mensajes claros que informan del resultado de una acción (confirmación, error o advertencia).
- Comportamientos similares para acciones equivalentes en distintas pantallas.
- Uso de animaciones y transiciones suaves cuando es necesario, sin distraer al usuario.

Esta coherencia reduce la curva de aprendizaje y mejora la sensación de control por parte del usuario.

2.7.4 Prevención de errores y confirmación de acciones

En aquellas acciones que pueden tener consecuencias importantes (por ejemplo, eliminar registros, activar alertas o modificar datos sensibles), se han incorporado mecanismos de confirmación que evitan errores accidentales.

Estos mecanismos incluyen:

- Diálogos de confirmación antes de ejecutar acciones críticas.
- Mensajes de advertencia claramente diferenciados visualmente.
- Limitación del número de pasos necesarios para completar una acción, sin eliminar la seguridad.

Este enfoque equilibra la rapidez de uso con la protección frente a errores involuntarios.

2.8 RA1.h – Integración global de la aplicación

La integración global de la aplicación es un aspecto fundamental para garantizar que todas las pantallas, componentes y funcionalidades trabajen de forma conjunta y coherente. En la aplicación Hamparo se ha prestado especial atención a que ninguna pantalla funcione de manera aislada, sino que todas formen parte de un flujo de uso lógico y bien definido.

Desde el inicio de la aplicación hasta las funcionalidades más específicas, el usuario puede desplazarse entre las distintas secciones de forma fluida, sin saltos inesperados ni pantallas desconectadas del contexto general.

2.8.1 Integración de pantallas y navegación

Todas las pantallas de la aplicación están integradas mediante un sistema de navegación centralizado, que permite controlar de forma clara los flujos de entrada y salida de cada sección. Esta integración garantiza que:

- El usuario siempre puede identificar en qué parte de la aplicación se encuentra.
- Las transiciones entre pantallas mantienen un contexto coherente.
- No existen accesos directos a pantallas que rompan la lógica de uso.

La navegación se ha diseñado teniendo en cuenta los distintos perfiles de usuario, adaptando el acceso a determinadas funcionalidades según el rol seleccionado.

2.8.2 Integración de componentes reutilizables

Los componentes reutilizables desarrollados se utilizan de forma consistente en distintas pantallas de la aplicación, lo que refuerza la coherencia visual y funcional del proyecto. Botones, tarjetas, listas y elementos interactivos mantienen el mismo comportamiento y apariencia independientemente de la pantalla en la que se encuentren.

Esta integración evita inconsistencias visuales y mejora la experiencia de usuario, ya que las acciones y elementos se comportan siempre de la misma manera.

2.8.3 Integración del estado y la lógica de interacción

La gestión del estado de la interfaz está correctamente integrada con la lógica de navegación y los eventos de usuario. Los cambios de estado provocados por acciones del usuario se reflejan de forma inmediata en la interfaz, manteniendo una relación clara entre acción y resultado.

Este enfoque evita situaciones en las que la interfaz no se actualiza correctamente o muestra información desfasada, contribuyendo a una experiencia de uso estable y fiable.

2.8.4 Estabilidad y comportamiento global

Durante las pruebas realizadas, la aplicación ha mostrado un comportamiento estable, sin errores críticos ni bloqueos durante el uso normal. Las distintas funcionalidades interactúan correctamente entre sí, incluso en pantallas con mayor carga de información o interacción.

La estabilidad general del proyecto es el resultado de una integración cuidadosa de todas las partes que componen la aplicación, desde la estructura de la interfaz hasta la gestión de eventos y datos.

CAPÍTULO 3 – RA2: NUEVAS FORMAS DE INTERACCIÓN

El presente capítulo desarrolla el **Resultado de Aprendizaje RA2**, centrado en el análisis y la propuesta de **Nuevas Interfaces de Usuario (NUI)** aplicadas a la aplicación Hamparo. Estas nuevas formas de interacción permiten ampliar las posibilidades de uso de la aplicación, adaptándola a contextos reales en los que la interacción tradicional mediante pantalla táctil puede no ser suficiente.

En aplicaciones con una finalidad social, sanitaria o asistencial, como es el caso de Hamparo, resulta especialmente relevante explorar alternativas de interacción que mejoren la accesibilidad, reduzcan el esfuerzo del usuario y permitan una respuesta rápida en situaciones de urgencia o limitación física.

Nota previa sobre el alcance del RA2

Las interacciones descritas en este capítulo se plantean **a nivel de diseño conceptual y como propuestas de mejora futura**. No todas las funcionalidades NUI analizadas se encuentran implementadas en la versión actual de la aplicación, pero su estudio y justificación se realiza siguiendo criterios realistas propios de un entorno profesional de desarrollo de software.

Este enfoque es habitual en proyectos reales, donde no todas las ideas se implementan en una primera versión, pero sí se analizan y documentan correctamente.

3.1 RA2.a – Herramientas de Nuevas Interfaces de Usuario (NUI)

3.1.1 Concepto de Nuevas Interfaces de Usuario

Las Nuevas Interfaces de Usuario (NUI) engloban un conjunto de técnicas y tecnologías que permiten al usuario interactuar con una aplicación de forma más natural, intuitiva y adaptada al contexto. A diferencia de las interfaces gráficas tradicionales (GUI), las NUI reducen la dependencia de controles visuales clásicos como botones o menús, y aprovechan capacidades avanzadas del dispositivo.

Estas interfaces pueden basarse en:

- Voz
- Gestos
- Movimiento corporal
- Reconocimiento facial
- Sensores biométricos
- Realidad aumentada

Su objetivo principal es **mejorar la accesibilidad y la eficiencia de la interacción**, especialmente en situaciones donde el uso convencional de una pantalla táctil resulta incómodo o inviable.

3.1.2 Herramientas NUI disponibles en el ecosistema Android

El sistema operativo Android ofrece un amplio conjunto de herramientas y APIs que permiten la implementación de NUI de forma relativamente accesible para los desarrolladores. Entre las más relevantes se encuentran:

- **Reconocimiento de voz:** APIs que permiten convertir la voz del usuario en texto y ejecutar acciones a partir de comandos hablados.
- **Síntesis de voz:** generación de respuestas habladas por parte de la aplicación.
- **Gestos avanzados:** detección de gestos complejos más allá de la pulsación simple.
- **Sensores del dispositivo:** acelerómetro, giroscopio, sensores de proximidad y movimiento.
- **Cámara y visión artificial:** reconocimiento facial o detección de expresiones.
- **Autenticación biométrica:** huella dactilar o reconocimiento facial para acceso seguro.
- **Tecnologías de realidad aumentada:** superposición de información digital sobre el entorno real.

Estas herramientas permiten diseñar experiencias de usuario más ricas y adaptadas a distintos perfiles y contextos de uso.

3.1.3 Justificación del uso de NUI en Hamparo

La aplicación Hamparo está orientada a un contexto asistencial, donde el usuario puede encontrarse en situaciones de estrés, urgencia o con limitaciones físicas. En este tipo de escenarios, las NUI aportan un valor añadido significativo.

Algunas de las razones por las que las NUI resultan especialmente adecuadas para Hamparo son:

- Posibilidad de interacción sin necesidad de mirar la pantalla.
- Reducción del número de pasos necesarios para ejecutar acciones críticas.
- Mejora de la accesibilidad para usuarios con dificultades motoras o visuales.
- Mayor rapidez de respuesta en situaciones de emergencia.

Por estos motivos, el análisis y la propuesta de NUI se consideran una ampliación lógica y coherente del proyecto.

3.2 RA2.b – Diseño conceptual de la interacción NUI

El diseño conceptual de la interacción mediante Nuevas Interfaces de Usuario (NUI) en la aplicación Hamparo se ha realizado siguiendo un enfoque centrado en el usuario (*user-centered design*). Este enfoque prioriza las necesidades reales de las personas que utilizarán la aplicación, teniendo en cuenta su contexto, limitaciones y objetivos.

En lugar de diseñar interacciones basadas únicamente en las capacidades técnicas del dispositivo, se ha puesto el foco en cómo las NUI pueden mejorar la experiencia de uso en situaciones reales, especialmente en contextos asistenciales y de cuidado.

3.2.1 Enfoque de diseño centrado en el usuario

El diseño conceptual de las NUI parte de la identificación de distintos perfiles de usuario que pueden interactuar con la aplicación, tales como cuidadores, familiares o personas dependientes. Cada uno de estos perfiles puede encontrarse en situaciones diferentes, lo que condiciona la forma más adecuada de interacción.

Algunos factores clave tenidos en cuenta en este diseño son:

- Nivel de estrés del usuario en determinadas situaciones.
- Posibles limitaciones físicas o cognitivas.
- Necesidad de ejecutar acciones de forma rápida.
- Entornos de uso con distracciones o ruido.

Este análisis permite diseñar interacciones más naturales y reducir la carga cognitiva del usuario.

3.2.2 Integración progresiva de las NUI

Las Nuevas Interfaces de Usuario se han planteado como un **complemento a la interfaz tradicional**, no como un sustituto obligatorio. Este planteamiento resulta fundamental para evitar el rechazo por parte de usuarios menos familiarizados con tecnologías avanzadas.

En el diseño conceptual de Hamparo:

- La interfaz táctil sigue siendo el método principal de interacción.
- Las NUI se activan de forma opcional o contextual.
- El usuario mantiene siempre el control sobre cómo interactuar con la aplicación.

Este enfoque híbrido permite beneficiarse de las ventajas de las NUI sin comprometer la usabilidad básica de la aplicación.

3.2.3 Escenarios de uso definidos

Durante el diseño conceptual se han definido distintos escenarios de uso que justifican la aplicación de NUI:

- **Situaciones de emergencia**, donde el usuario necesita activar una función sin navegar por menús.
- **Uso con una sola mano**, por ejemplo, cuando el usuario está asistiendo a otra persona.
- **Limitaciones visuales temporales**, como poca iluminación o dificultad para enfocar la pantalla.
- **Usuarios con movilidad reducida**, que pueden tener problemas para interactuar con controles pequeños.

Cada uno de estos escenarios ha influido directamente en la propuesta de interacciones alternativas.

3.2.4 Diseño de flujos de interacción alternativos

A partir de los escenarios definidos, se han diseñado flujos de interacción alternativos que reducen el número de pasos necesarios para realizar acciones clave. Estos flujos se caracterizan por:

- Activación rápida de funciones críticas.
- Confirmaciones claras para evitar errores.
- Retroalimentación inmediata al usuario.

Por ejemplo, una acción que normalmente requeriría varios pasos en la interfaz táctil podría activarse mediante un único comando de voz o gesto predefinido.

3.2.5 Accesibilidad como eje del diseño

La accesibilidad ha sido un eje central en el diseño conceptual de las NUI. El objetivo no es únicamente facilitar el uso a personas con discapacidad, sino mejorar la experiencia de uso para todos los usuarios.

Algunos principios aplicados en este diseño son:

- Reducción de la precisión necesaria para interactuar.
- Uso de mensajes claros y directos.
- Evitar sobrecarga de información visual.
- Posibilidad de recibir feedback auditivo o visual según el contexto.

Este enfoque garantiza que las NUI aporten un valor real y no se conviertan en una barrera adicional.

3.2.6 Coherencia con el diseño general de la aplicación

Todas las propuestas de interacción NUI se han diseñado manteniendo coherencia con la identidad visual y funcional de la aplicación Hamparo. Esto implica que:

- Las acciones activadas mediante NUI generan el mismo resultado que las acciones táctiles.
- Los mensajes y respuestas mantienen el mismo tono y estilo.
- No se introducen comportamientos inesperados para el usuario.

De este modo, las NUI se integran de forma natural dentro del diseño global de la aplicación.

3.3 RA2.c – Interacción por voz

La interacción por voz constituye una de las formas más naturales de comunicación entre el ser humano y la tecnología. En el contexto de aplicaciones móviles con una finalidad asistencial, como Hamparo, la voz se convierte en un recurso especialmente valioso, ya que permite al usuario interactuar con la aplicación sin necesidad de utilizar las manos ni fijar la vista en la pantalla.

Por este motivo, la interacción por voz se ha analizado como una de las principales propuestas de Nuevas Interfaces de Usuario dentro del proyecto.

3.3.1 Justificación del uso de la voz como interfaz

La voz es una interfaz intuitiva que reduce significativamente la carga cognitiva del usuario. En situaciones de estrés, urgencia o limitación física, la capacidad de ejecutar acciones mediante comandos hablados puede marcar una diferencia importante en la experiencia de uso.

En el caso de Hamparo, la interacción por voz resulta especialmente adecuada por las siguientes razones:

- Permite actuar de forma rápida sin navegar por menús.
- Facilita el uso de la aplicación a personas con dificultades motoras.
- Reduce errores derivados de pulsaciones imprecisas.
- Puede utilizarse incluso con el dispositivo bloqueado o fuera del campo visual.

Estas ventajas convierten la voz en un complemento ideal a la interfaz táctil tradicional

3.3.2 Herramientas de reconocimiento de voz en Android

El ecosistema Android proporciona herramientas maduras para la implementación de reconocimiento de voz, lo que hace viable su integración desde un punto de vista técnico. Estas herramientas permiten:

- Convertir la voz del usuario en texto.
- Analizar comandos hablados.
- Ejecutar acciones en función del contenido reconocido.

Además, Android ofrece compatibilidad con distintos idiomas y modelos de reconocimiento, lo que mejora la accesibilidad y la adaptación a distintos perfiles de usuario.

Aunque la interacción por voz no se encuentra completamente implementada en la versión actual de la aplicación, su análisis se ha realizado teniendo en cuenta estas capacidades reales del sistema operativo.

3.3.3 Casos de uso propuestos para la interacción por voz

Durante el diseño conceptual se han definido distintos casos de uso en los que la interacción por voz aportaría un valor añadido claro dentro de la aplicación Hamparo:

- Activación de una alerta de emergencia mediante un comando corto y predefinido.
- Consulta del estado de una alerta activa.

- Solicitud de ayuda contextual o información básica.
- Confirmación de acciones sin necesidad de interacción táctil.

Estos casos de uso se han seleccionado por su relevancia en situaciones donde el tiempo de reacción es crítico.

3.3.4 Diseño del flujo de interacción por voz

El flujo de interacción por voz se ha diseñado para ser simple, claro y seguro. De forma general, el proceso seguiría los siguientes pasos:

1. Activación del sistema de reconocimiento de voz.
2. Captura del comando hablado por el usuario.
3. Análisis y validación del comando.
4. Ejecución de la acción asociada.
5. Confirmación de la acción mediante feedback visual o auditivo.

Este flujo reduce la probabilidad de errores y permite al usuario confirmar que la acción se ha ejecutado correctamente.

3.3.5 Feedback al usuario

Un aspecto fundamental en la interacción por voz es el feedback proporcionado al usuario. En el diseño conceptual de Hamparo se ha considerado que cada acción activada por voz debe ir acompañada de una respuesta clara.

Este feedback puede adoptar distintas formas:

- Mensajes hablados generados por la aplicación.
- Mensajes visuales en pantalla.
- Indicadores sonoros.

El objetivo es evitar situaciones en las que el usuario no sepa si la aplicación ha entendido correctamente el comando.

3.3.6 Limitaciones y riesgos de la interacción por voz

A pesar de sus ventajas, la interacción por voz presenta ciertas limitaciones que han sido tenidas en cuenta durante el diseño conceptual:

- Presencia de ruido ambiental.
- Errores de reconocimiento en determinados acentos o pronunciaciones.
- Problemas de privacidad en entornos públicos.

- Dependencia de conexión a internet en algunos casos.

Por este motivo, la interacción por voz se plantea siempre como una **opción complementaria**, nunca como el único medio de interacción disponible.

3.3.7 Seguridad y control del usuario

En acciones sensibles, como la activación de alertas o el acceso a información personal, se ha considerado la necesidad de incorporar mecanismos de seguridad adicionales. Estos pueden incluir:

- Confirmación explícita del comando.
- Uso combinado con autenticación biométrica.
- Limitación de comandos disponibles por voz.

De esta forma se garantiza que la interacción por voz no comprometa la seguridad del usuario.

3.4 RA2.d – Interacción por gestos

La interacción por gestos constituye una de las formas más habituales y naturales de interacción en dispositivos móviles. A diferencia de otros tipos de Nuevas Interfaces de Usuario, los gestos no requieren hardware adicional ni un aprendizaje complejo, ya que forman parte del uso cotidiano de los smartphones.

En el contexto de la aplicación Hamparo, la interacción por gestos se plantea como un complemento ideal a la interfaz tradicional, permitiendo ejecutar acciones de forma rápida, intuitiva y con un menor esfuerzo cognitivo.

3.4.1 Concepto de interacción por gestos

La interacción por gestos se basa en la interpretación de movimientos táctiles realizados por el usuario sobre la pantalla, como deslizamientos, pulsaciones prolongadas o combinaciones de gestos. Estos movimientos pueden asociarse a acciones específicas dentro de la aplicación.

A diferencia de los botones tradicionales, los gestos permiten reducir la saturación visual de la interfaz y agilizar determinadas acciones frecuentes.

3.4.2 Justificación del uso de gestos en Hamparo

La aplicación Hamparo está diseñada para ser utilizada en situaciones donde el usuario puede necesitar rapidez y simplicidad. En este tipo de contextos, los gestos aportan varias ventajas:

- Permiten ejecutar acciones con un solo movimiento.
- Reducen la necesidad de buscar botones pequeños en pantalla.
- Facilitan el uso con una sola mano.
- Resultan naturales para la mayoría de los usuarios de dispositivos móviles.

Estas características hacen que los gestos sean especialmente adecuados para una aplicación con una finalidad asistencial.

3.4.3 Gestos propuestos y su funcionalidad

Durante el diseño conceptual se han propuesto gestos simples y fácilmente reconocibles, evitando combinaciones complejas que puedan generar confusión. Algunos ejemplos de gestos planteados son:

- **Deslizamiento horizontal** para navegar entre secciones o pantallas relacionadas.
- **Deslizamiento vertical** para desplazarse por listas de información.
- **Pulsación prolongada** para mostrar opciones adicionales o acciones secundarias.
- **Gestos de confirmación** en acciones importantes, combinados con feedback visual.

Estos gestos siguen patrones comunes en aplicaciones móviles, lo que reduce la curva de aprendizaje.

3.4.4 Diseño de la experiencia de usuario basada en gestos

El diseño de la interacción por gestos se ha realizado teniendo en cuenta la necesidad de mantener una experiencia de usuario clara y predecible. Para ello se han aplicado los siguientes principios:

- Cada gesto tiene una única función claramente definida.
- Los gestos no sustituyen acciones críticas sin confirmación.
- Siempre existe una alternativa táctil visible para la misma acción.

Este enfoque evita que el usuario dependa exclusivamente de gestos ocultos o difíciles de descubrir.

3.4.5 Accesibilidad y gestos

La accesibilidad ha sido un factor clave en el diseño de la interacción por gestos. Aunque los gestos pueden mejorar la rapidez de uso, también pueden suponer una barrera para determinados usuarios si no se diseñan correctamente.

Por este motivo:

- Se han priorizado gestos simples y amplios.
- Se evita la necesidad de movimientos precisos.
- Se mantiene compatibilidad con controles tradicionales.

De esta forma, la interacción por gestos mejora la experiencia sin excluir a usuarios con limitaciones motoras o cognitivas.

3.4.6 Prevención de errores mediante gestos

En acciones sensibles, como eliminar información o activar funciones importantes, los gestos se complementan con mecanismos de prevención de errores. Estos mecanismos incluyen:

- Confirmaciones visuales antes de ejecutar la acción.
- Posibilidad de cancelar la acción.
- Feedback inmediato tras el gesto realizado.

Este diseño reduce el riesgo de errores accidentales y aumenta la confianza del usuario.

3.4.7 Integración con el resto de la interfaz

Los gestos propuestos se integran de forma coherente con el resto de la interfaz de la aplicación Hamparo. No se introducen gestos que contradigan el comportamiento de los controles tradicionales, sino que ambos métodos se refuerzan mutuamente.

El resultado es una experiencia de uso fluida, natural y consistente en todas las pantallas de la aplicación.

3.5 RA2.e – Detección facial y corporal

La detección facial y corporal es una de las áreas más avanzadas dentro de las Nuevas Interfaces de Usuario. Mediante el uso de la cámara del dispositivo y técnicas de visión por ordenador, es posible reconocer rostros, detectar presencia, analizar expresiones o incluso interpretar posturas corporales.

En aplicaciones asistenciales, como Hamparo, este tipo de tecnologías puede aportar un valor añadido importante en términos de **seguridad, supervisión y personalización de la experiencia de usuario**, siempre que se utilicen de forma responsable y respetando la privacidad.

3.5.1 Concepto y posibilidades

La detección facial y corporal permite que la aplicación “entienda” mejor el contexto físico en el que se encuentra el usuario. Algunas de las posibilidades que ofrece son:

- Reconocer si una persona está presente frente al dispositivo.
- Verificar la identidad del usuario mediante rasgos faciales.
- Detectar si ha habido un cambio brusco en la postura (por ejemplo, una caída).
- Ajustar la interfaz en función de la distancia o el ángulo de visión del usuario.

Estas capacidades convierten al dispositivo móvil en algo más que una simple pantalla táctil, permitiendo una interacción más adaptada al entorno real.

3.5.2 Aplicación potencial en Hamparo

En el contexto de la aplicación Hamparo, la detección facial y corporal podría aplicarse de varias formas, especialmente orientadas a mejorar la seguridad y el seguimiento de la persona dependiente. A nivel conceptual, se plantean las siguientes posibilidades:

- **Verificación de identidad:** utilizar reconocimiento facial para asegurar que el usuario que accede a cierta información sensible (por ejemplo, historial médico o configuración avanzada) es realmente la persona autorizada.
- **Detección de inactividad o ausencia:** identificar si el dispositivo permanece sin uso prolongado en determinados momentos del día, lo que podría servir como indicio de que el paciente no está siguiendo las pautas normales.
- **Apoyo en la detección de posibles caídas** (combinado con otros sensores): aunque una caída se suele detectar mejor con acelerómetros, la cámara podría utilizarse como mecanismo complementario para confirmar la situación.

Estas ideas no están implementadas en la versión actual, pero se han tenido en cuenta como posibles líneas de evolución futura del proyecto.

3.5.3 Beneficios y riesgos

El uso de detección facial y corporal presenta una serie de beneficios claros, pero también riesgos que deben ser gestionados adecuadamente.

Beneficios potenciales:

- Mayor seguridad en el acceso a información sensible.
- Supervisión más precisa del uso de la aplicación.
- Adaptación de la interfaz al contexto físico del usuario.
- Posibilidad de automatizar ciertas comprobaciones sin intervención manual.

Riesgos y desafíos:

- Necesidad de gestionar correctamente los permisos de cámara.
- Posibles preocupaciones del usuario relacionadas con la privacidad.
- Dependencia de condiciones de iluminación adecuadas.
- Necesidad de cumplir la normativa sobre protección de datos (especialmente si se almacena información biométrica).

Por ello, cualquier uso real de estas tecnologías debe acompañarse de medidas claras de transparencia y consentimiento informado por parte del usuario.

3.5.4 Aspectos éticos y de privacidad

Un punto clave en el uso de detección facial y corporal es el respeto a la privacidad y a la normativa vigente. En el diseño conceptual de Hamparo se asume que:

- El usuario debe ser informado de forma clara si se utiliza la cámara para algún tipo de detección.
- No se almacenarán imágenes ni datos biométricos sin consentimiento explícito.
- El procesamiento debería realizarse, siempre que sea posible, de forma local en el dispositivo para reducir riesgos.

Desde el punto de vista ético, se considera que la detección facial solo debería utilizarse cuando aporte un beneficio real al usuario y nunca como un requisito obligatorio para acceder a la aplicación.

3.5.5 Integración con el resto de la aplicación

Si en futuras versiones se implementase detección facial o corporal en Hamparo, su integración debería:

- Mantener coherencia con el sistema de autenticación ya existente (por ejemplo, combinándose con PIN o credenciales).
- Evitar añadir pasos innecesarios en el uso diario de la aplicación.
- Limitarse a escenarios bien justificados, como acceso a secciones críticas o verificación puntual de identidad.

La detección facial no debe convertirse en una barrera adicional, sino en una opción que refuerce la seguridad o mejore la experiencia de uso en situaciones concretas.

3.6 RA2.f – Realidad aumentada

La realidad aumentada (RA) es una tecnología que permite superponer información digital sobre el entorno físico del usuario en tiempo real, utilizando la cámara del dispositivo móvil como medio principal de interacción. A diferencia de la realidad virtual, la RA no sustituye el entorno real, sino que lo complementa con elementos informativos adicionales.

En el ámbito de las aplicaciones móviles con finalidad social o asistencial, la realidad aumentada puede convertirse en una herramienta de gran utilidad para guiar, informar y asistir al usuario de forma visual e intuitiva.

3.6.1 Concepto de realidad aumentada y su evolución

La realidad aumentada ha experimentado un gran desarrollo en los últimos años gracias al aumento de la potencia de los dispositivos móviles y la mejora de las herramientas de desarrollo. Actualmente, es posible implementar experiencias de RA directamente en smartphones sin necesidad de dispositivos adicionales.

Esta tecnología se utiliza cada vez más en ámbitos como:

- Educación y formación.
- Salud y rehabilitación.
- Asistencia técnica.
- Orientación y navegación.
- Concienciación social.

Su versatilidad la convierte en una opción interesante para futuras ampliaciones de la aplicación Hamparo.

3.6.2 Herramientas de realidad aumentada en Android

El ecosistema Android ofrece herramientas específicas para el desarrollo de aplicaciones con realidad aumentada, como plataformas que permiten:

- Detectar superficies y espacios reales.
- Posicionar objetos virtuales en el entorno.
- Mantener la estabilidad de los elementos mostrados.
- Adaptar la experiencia a distintos dispositivos.

Estas herramientas permiten diseñar experiencias de RA realistas y técnicamente viables, siempre que se disponga de los recursos necesarios.

3.6.3 Aplicación conceptual de la realidad aumentada en Hamparo

Aunque la realidad aumentada no se encuentra implementada en la versión actual de la aplicación, se ha analizado su posible aplicación como una mejora futura. A nivel conceptual, la RA podría utilizarse en Hamparo para:

- **Guiar visualmente al usuario** hacia zonas seguras dentro del entorno.
- Mostrar información contextual sobre objetos o espacios relevantes.
- Apoyar a cuidadores o familiares en la identificación de recursos cercanos.
- Facilitar instrucciones visuales en situaciones donde el texto resulta insuficiente.

Estas aplicaciones resultarían especialmente útiles en entornos desconocidos o en situaciones de estrés.

3.6.4 Ventajas de la realidad aumentada en aplicaciones asistenciales

La incorporación de realidad aumentada puede aportar varias ventajas a aplicaciones con una finalidad asistencial:

- Mejora de la comprensión visual de la información.
- Reducción de la necesidad de interpretar textos complejos.
- Interacción más intuitiva y directa con el entorno.
- Mayor implicación del usuario en el uso de la aplicación.

Estas ventajas hacen que la RA sea una herramienta especialmente interesante para usuarios con dificultades cognitivas o problemas de comprensión lectora.

3.6.5 Limitaciones técnicas y de usabilidad

A pesar de sus ventajas, la realidad aumentada presenta también una serie de limitaciones que deben tenerse en cuenta:

- Requiere dispositivos con hardware compatible.
- Puede consumir más batería que otras funcionalidades.
- Depende de condiciones adecuadas de iluminación.
- Puede resultar compleja para ciertos perfiles de usuario si no se diseña correctamente.

Por este motivo, su uso debe plantearse siempre de forma opcional y justificada, evitando convertirla en un requisito obligatorio para el uso de la aplicación.

3.6.6 Consideraciones de accesibilidad y seguridad

Desde el punto de vista de la accesibilidad, es importante que la realidad aumentada no sustituya por completo otros métodos de interacción. En el diseño conceptual de Hamparo se considera que:

- La RA debe complementarse con información textual o auditiva.
- El usuario debe poder abandonar la experiencia de RA en cualquier momento.
- Deben evitarse estímulos visuales excesivos que puedan causar confusión.

En cuanto a la seguridad, la aplicación debe advertir al usuario de que mantenga la atención en su entorno físico mientras utiliza funciones de realidad aumentada, especialmente en espacios abiertos.

3.7 Comparativa entre interfaz tradicional y NUI

Aspecto	Interfaz tradicional	NUI (Natural User Interface)
Dependencia visual	Alta	Media / Baja
Velocidad de interacción	Media	Alta
Accesibilidad	Limitada	Alta
Curva de aprendizaje	Baja	Media

Esta comparativa refuerza la idoneidad del enfoque adoptado.

CAPÍTULO 4 – RA3: COMPONENTES REUTILIZABLES

El presente capítulo desarrolla el **Resultado de Aprendizaje RA3**, centrado en la identificación, diseño, creación y uso de **componentes reutilizables** dentro de la aplicación Hamparo. Este RA es clave para demostrar una correcta aplicación de buenas prácticas de desarrollo de interfaces, orientadas a la modularidad, mantenibilidad y escalabilidad del proyecto.

En aplicaciones reales, el uso de componentes reutilizables permite reducir la duplicación de código, mejorar la coherencia visual y facilitar futuras ampliaciones o modificaciones. Por este motivo, este aspecto ha sido tratado de forma prioritaria durante el desarrollo del proyecto.

4.1 RA3.a – Herramientas para la creación de componentes

4.1.1 Componentes en Jetpack Compose

Jetpack Compose está diseñado de forma nativa para favorecer la creación de componentes reutilizables mediante funciones anotadas con `@Composable`. Cada componente representa una unidad independiente de interfaz que puede reutilizarse en distintas pantallas y contextos.

En el proyecto Hamparo, esta filosofía ha permitido estructurar la interfaz en pequeños bloques funcionales, facilitando la lectura del código y la reutilización de elementos comunes.

4.1.2 Ventajas frente a enfoques tradicionales

En comparación con el enfoque tradicional basado en XML, el uso de componentes en Jetpack Compose aporta múltiples ventajas:

- Eliminación de la necesidad de inflar vistas.
- Menor acoplamiento entre lógica y diseño.
- Mayor facilidad para reutilizar elementos visuales.
- Mejor adaptación a cambios de diseño.

Estas ventajas hacen que Jetpack Compose sea especialmente adecuado para proyectos de tamaño medio o grande, como el desarrollado en este trabajo.

4.1.3 Organización del código por componentes

Los componentes reutilizables se han organizado en paquetes específicos dentro del proyecto, diferenciándolos claramente de las pantallas completas. Esta organización permite:

- Localizar rápidamente los componentes comunes.
- Evitar duplicación innecesaria de código.
- Facilitar el mantenimiento y la evolución del proyecto.

Esta estructura es habitual en proyectos profesionales y mejora la escalabilidad del código.

4.2 RA3.b – Diseño y creación de componentes reutilizables

4.2.1 Identificación de elementos reutilizables

Durante el análisis de la interfaz de la aplicación Hamparo se identificaron varios elementos que aparecían de forma recurrente en distintas pantallas. Entre ellos destacan:

- Botones de acción principal.
- Tarjetas informativas.
- Listas de datos.
- Diálogos de confirmación.
- Componentes gráficos personalizados.

Estos elementos fueron abstraídos en componentes reutilizables para evitar repetir código y mantener coherencia visual.

4.2.2 Ejemplos de componentes creados

En el proyecto se han desarrollado distintos componentes reutilizables, entre los que se pueden destacar:

- Componentes de tipo **Card** para mostrar información estructurada.
- Botones personalizados con estilos coherentes.
- Componentes gráficos como gráficos de datos o indicadores visuales.
- Contenedores reutilizables para listas y secciones.

Cada componente encapsula tanto su diseño visual como su comportamiento básico, permitiendo su uso en diferentes contextos.

4.2.3 Beneficios obtenidos

El uso de componentes reutilizables ha permitido:

- Reducir significativamente la duplicación de código.
- Mantener una identidad visual uniforme.
- Simplificar la modificación de estilos globales.
- Facilitar la lectura y comprensión del código.

Estos beneficios son especialmente importantes en proyectos que pueden evolucionar con el tiempo.

4.3 RA3.c – Parámetros y valores por defecto

4.3.1 Parametrización de componentes

Los componentes reutilizables se han diseñado para aceptar parámetros que permiten personalizar su comportamiento y apariencia sin necesidad de duplicarlos. Algunos de los parámetros utilizados incluyen:

- Textos mostrados.
- Acciones asociadas a eventos.
- Estados habilitados/deshabilitados.
- Colores o estilos específicos.

Esta parametrización hace que los componentes sean flexibles y adaptables a distintos escenarios.

4.3.2 Uso de valores por defecto

Para facilitar el uso de los componentes y reducir errores, se han definido valores por defecto para muchos de sus parámetros. Esto permite utilizar el componente con una configuración mínima cuando no es necesario personalizarlo.

El uso de valores por defecto mejora la experiencia del desarrollador y reduce la complejidad del código.

4.4 RA3.d – Gestión de eventos en componentes

4.4.1 Asociación de eventos mediante callbacks

Los componentes gestionan los eventos de interacción mediante callbacks, siguiendo el principio de inversión de control. De este modo, el componente notifica cuando ocurre un evento, pero no decide qué acción ejecutar.

Este enfoque desacopla el componente de la lógica de negocio y facilita su reutilización en distintos contextos.

4.4.2 Fluidez de la interacción

Gracias a la correcta gestión de eventos, los componentes responden de forma inmediata a las acciones del usuario, manteniendo una interacción fluida y coherente en toda la aplicación.

4.5 RA3.f – Documentación de componentes

4.5.1 Importancia de la documentación

La documentación de los componentes reutilizables es fundamental para facilitar su uso y mantenimiento. En un entorno profesional, esta documentación permite que otros desarrolladores comprendan rápidamente cómo utilizar cada componente.

La documentación de los componentes es fundamental para:

- Facilitar su reutilización.
- Mejorar la comprensión del código.
- Permitir futuras ampliaciones.

4.5.2 Documentación aplicada en el proyecto

En el proyecto Hamparo se ha documentado cada componente indicando:

- Su propósito.
- Los parámetros disponibles.
- El comportamiento esperado.
- Ejemplos de uso cuando es necesario.

Esta documentación contribuye a la claridad y calidad global del código.

4.6 RA3.h – Integración de componentes en la aplicación

4.6.1 Uso en múltiples pantallas

Los componentes reutilizables se utilizan en diversas pantallas de la aplicación, garantizando una experiencia de usuario consistente. Esto refuerza la coherencia visual y funcional del proyecto.

4.6.2 Impacto en la calidad del proyecto

La integración de componentes reutilizables ha tenido un impacto positivo directo en la calidad del proyecto, mejorando la mantenibilidad, la escalabilidad y la estabilidad general de la aplicación.

Gracias a esta integración se ha conseguido:

- Reducción de código duplicado.
- Mayor facilidad de mantenimiento.
- Mejora de la calidad global del proyecto.

CAPÍTULO 5 – RA4: USABILIDAD Y DISEÑO VISUAL

El presente capítulo desarrolla el **Resultado de Aprendizaje RA4**, centrado en la aplicación de principios de **usabilidad, accesibilidad y diseño visual** en la interfaz de la aplicación Hamparo. Este resultado evalúa no solo el aspecto estético de la aplicación, sino también la claridad de uso, la organización de los elementos, la elección de controles y la adecuación de la interfaz a distintos perfiles de usuario.

Dado el carácter asistencial de la aplicación, el diseño visual y la usabilidad han sido considerados elementos clave desde las primeras fases del desarrollo.

5.1 RA4.a – Aplicación de estándares de diseño

5.1.1 Estándares de diseño en Android

Para el diseño de la interfaz de la aplicación Hamparo se han tenido en cuenta los estándares oficiales de diseño de Android, principalmente los establecidos por **Material Design**. Estos estándares proporcionan una base sólida para crear interfaces coherentes, accesibles y fácilmente reconocibles por los usuarios.

El uso de estándares de diseño permite:

- Reducir la curva de aprendizaje del usuario.
- Mantener coherencia visual entre pantallas.
- Garantizar un comportamiento predecible de los controles.
- Mejorar la accesibilidad de la interfaz.

5.1.2 Aplicación práctica de los estándares en Hamparo

En la aplicación Hamparo, los estándares de diseño se reflejan en múltiples aspectos de la interfaz, tales como:

- Uso de componentes Material (botones, tarjetas, diálogos).
- Jerarquía visual clara entre títulos, textos y acciones.
- Espaciados consistentes entre elementos.
- Comportamientos esperados al interactuar con los controles.

Estos principios se han aplicado de forma coherente en todas las pantallas de la aplicación.

5.1.3 Adaptación de los estándares al contexto del proyecto

Aunque los estándares de diseño ofrecen una guía general, no siempre se aplican de forma estricta. En Hamparo se ha realizado una adaptación consciente de estos estándares para responder mejor a las necesidades del usuario final.

Por ejemplo:

- Se han utilizado botones de mayor tamaño para facilitar la interacción.
- Se ha reducido la densidad de información en pantalla.
- Se ha priorizado la claridad sobre la complejidad visual.

Estas decisiones están alineadas con el contexto asistencial de la aplicación.

5.2 RA4.b – Valoración y justificación de los estándares

5.2.1 Análisis crítico de los estándares utilizados

El uso de estándares de diseño no debe ser automático. En este proyecto se ha realizado una valoración crítica de cada decisión de diseño, analizando si realmente mejora la experiencia de usuario o si puede generar confusión.

Este análisis ha permitido adaptar las guías generales a un contexto específico, evitando soluciones genéricas que no aportan valor real.

5.2.2 Decisiones de diseño justificadas

Algunas de las decisiones de diseño justificadas en el proyecto son:

- Priorizar acciones principales frente a opciones secundarias.
- Utilizar iconografía clara y reconocible.
- Evitar menús complejos o profundos.
- Reducir el número de acciones visibles simultáneamente.

Estas decisiones se han tomado pensando en la facilidad de uso y en la reducción de errores.

5.3 RA4.c – Diseño y uso de menús

5.3.1 Tipos de menús utilizados

La aplicación Hamparo utiliza menús simples y fácilmente comprensibles, evitando estructuras complejas que puedan dificultar la navegación. Los menús se han diseñado para ofrecer acceso directo a las funcionalidades principales.

Se han priorizado:

- Menús claros y poco profundos.
- Opciones bien etiquetadas.
- Accesos directos a acciones frecuentes.

5.3.2 Integración de los menús en la navegación

Los menús están integrados de forma coherente con el sistema de navegación general de la aplicación. Esto permite al usuario desplazarse entre secciones sin perder el contexto ni la orientación dentro de la app.

5.4 RA4.d – Distribución de acciones

5.4.1 Priorización de acciones principales

Las acciones más importantes se sitúan en zonas fácilmente accesibles de la pantalla, siguiendo principios de ergonomía móvil. Esto resulta especialmente importante en dispositivos que se utilizan con una sola mano.

5.4.2 Prevención de errores en la distribución

Para evitar errores accidentales, las acciones críticas se diferencian visualmente de las secundarias y, cuando es necesario, se acompañan de confirmaciones previas.

5.5 RA4.e – Distribución de controles

5.5.1 Organización y jerarquía visual

Los controles de la interfaz se organizan siguiendo una jerarquía visual clara, facilitando que el usuario identifique rápidamente qué acciones puede realizar y cuál es su importancia relativa.

5.5.2 Espaciado y alineación

Se ha cuidado el espaciado entre controles para mejorar la legibilidad y evitar pulsaciones involuntarias, especialmente en pantallas con interacción táctil.

5.6 RA4.f – Elección de controles

5.6.1 Selección adecuada de componentes

Cada tipo de control se ha elegido en función de su propósito:

- Botones para acciones directas.
- Switches para opciones activables.
- Diálogos para confirmaciones.
- Tarjetas para mostrar información estructurada.

5.6.2 Justificación de la elección

Esta elección evita ambigüedades y mejora la comprensión de la interfaz, contribuyendo a una experiencia de usuario más intuitiva.

5.7 RA4.g – Diseño visual global

5.7.1 Estética y legibilidad

El diseño visual de la aplicación busca un equilibrio entre estética y funcionalidad, priorizando siempre la legibilidad y la claridad de la información.

El diseño visual de Hamparo busca un equilibrio entre estética y funcionalidad:

- Colores sobrios.
- Contraste adecuado.
- Tipografía legible.

5.7.2 Coherencia visual

Todos los elementos visuales mantienen una coherencia en colores, tipografías y estilos, reforzando la identidad de la aplicación.

5.8 RA4.h – Claridad de mensajes

5.8.1 Lenguaje utilizado

Los mensajes mostrados en la aplicación utilizan un lenguaje claro, directo y comprensible, evitando tecnicismos innecesarios.

5.8.2 Adaptación al usuario

Los mensajes están diseñados para ser entendidos rápidamente, incluso en situaciones de estrés o urgencia.

5.9 RA4.i – Pruebas de usabilidad

5.9.1 Metodología de pruebas

Se han realizado pruebas de usabilidad básicas con distintos perfiles de usuario, observando su interacción con la aplicación y detectando posibles puntos de mejora.

5.9.2 Resultados obtenidos

Estas pruebas permitieron ajustar aspectos de la navegación, el tamaño de los controles y la claridad de algunos mensajes.

5.10 RA4.j – Evaluación en distintos dispositivos

5.10.1 Dispositivos utilizados

La aplicación se ha probado en distintos dispositivos y emuladores, con diferentes tamaños de pantalla y resoluciones.

5.10.2 Resultados de la evaluación

La interfaz se adapta correctamente a distintos formatos, manteniendo funcionalidad, legibilidad y coherencia visual.

CAPÍTULO 6 – RA5: INFORMES

El presente capítulo desarrolla el **Resultado de Aprendizaje RA5**, centrado en la **generación, estructuración y presentación de informes** a partir de datos dentro de una aplicación. En el contexto del proyecto Hamparo, los informes se plantean como una funcionalidad orientada principalmente a perfiles avanzados (por ejemplo, administradores o cuidadores), permitiendo analizar el uso de la aplicación y el estado de determinados registros de forma clara y visual.

En entornos profesionales, los informes no siempre se implementan completamente en una primera versión del software, pero sí se **diseñan, documentan y justifican** correctamente. Este enfoque es el que se ha seguido en el presente proyecto.

Nota previa sobre el alcance de los informes (RA5)

Los informes descritos en este capítulo se abordan desde un **enfoque conceptual y de diseño**, coherente con un entorno profesional real. Aunque no todos los informes se encuentran implementados de forma completa en la versión actual de la aplicación, su estructura, finalidad y posibles representaciones se han definido de manera clara y razonada.

Este planteamiento cumple con los criterios del RA5, que evalúan tanto la capacidad técnica como la comprensión del valor de los informes dentro de una aplicació

6.1 RA5.a – Estructura del informe

6.1.1 Concepto de informe en aplicaciones móviles

Un informe es una representación estructurada de datos relevantes cuyo objetivo principal es facilitar la interpretación de la información y apoyar la toma de decisiones. En aplicaciones móviles, los informes deben ser especialmente claros y visuales, ya que el espacio disponible es limitado y el tiempo de atención del usuario suele ser reducido.

En el caso de Hamparo, los informes se conciben como herramientas de apoyo para el seguimiento, la supervisión y el análisis del uso de la aplicación.

6.1.2 Estructura general de los informes en Hamparo

Los informes diseñados para la aplicación Hamparo siguen una estructura común que garantiza coherencia y facilidad de uso. Esta estructura se compone de los siguientes elementos:

- **Encabezado del informe**, donde se identifica el tipo de informe y su finalidad.
- **Filtros aplicables**, que permiten acotar los datos mostrados.
- **Datos principales**, presentados de forma clara y ordenada.
- **Valores calculados**, como totales, recuentos o medias.
- **Representación gráfica**, que facilita la comprensión visual.
- **Conclusión o interpretación**, orientada al usuario.

Esta estructura se ha definido para que los informes sean comprensibles incluso para usuarios sin conocimientos técnicos.

6.1.3 Justificación de la estructura propuesta

La estructura propuesta responde a criterios de usabilidad y claridad de la información. Separar los datos en bloques bien definidos evita la sobrecarga visual y permite que el usuario identifique rápidamente la información más relevante.

Además, esta estructura facilita la ampliación futura de los informes, ya que permite añadir nuevos bloques sin modificar el diseño general.

6.1.4 Coherencia con el diseño de la aplicación

Los informes se han diseñado manteniendo coherencia con el estilo visual y la navegación general de la aplicación Hamparo. Esto implica:

- Uso de los mismos colores y tipografías.
- Componentes visuales coherentes con el resto de las pantallas.
- Comportamientos de navegación similares.

Esta coherencia refuerza la sensación de integración de los informes dentro de la aplicación.

6.2 RA5.b – Generación de informes a partir de fuentes de datos

Los informes diseñados en la aplicación Hamparo se basan en la recopilación y el tratamiento de datos generados durante el uso normal de la aplicación. Estos datos constituyen la materia prima a partir de la cual se construyen los distintos informes, permitiendo analizar tanto el estado de la información registrada como el comportamiento general del sistema.

En un entorno profesional, la correcta identificación de las fuentes de datos es un paso clave para garantizar la fiabilidad y utilidad de los informes generados.

6.2.1 Identificación de las fuentes de datos

En el contexto de Hamparo, se han identificado varias fuentes de datos relevantes para la generación de informes, entre las que destacan:

- Registros introducidos por el usuario (mediciones, estados, eventos).
- Historial de acciones realizadas dentro de la aplicación.
- Datos asociados a perfiles de usuario.
- Información temporal (fechas y horas de registro).

Estas fuentes permiten obtener una visión global del uso y del estado de la aplicación

6.2.2 Tratamiento de los datos para la generación de informes

Antes de ser presentados en forma de informe, los datos deben ser tratados y organizados. Este tratamiento incluye:

- Filtrado de datos no válidos o incompletos.
- Ordenación cronológica de los registros.
- Agrupación de información por categorías o periodos de tiempo.
- Preparación de los datos para su representación visual.

Este proceso garantiza que los informes sean claros, coherentes y útiles para el usuario final.

Los informes se integran como pantallas específicas dentro de la aplicación, accesibles principalmente para perfiles con permisos avanzados, como administradores o cuidadores. Esta integración permite consultar la información sin salir de la aplicación, manteniendo una experiencia de usuario coherente.

La generación de informes a partir de datos internos refuerza el carácter funcional de la aplicación y amplía sus posibilidades de uso en contextos reales.

6.3 RA5.c – Filtros sobre los valores a presentar

En los informes diseñados para la aplicación Hamparo se contempla el uso de **filtros** como un elemento fundamental para facilitar la interpretación de la información. Los filtros permiten reducir la cantidad de datos mostrados y centrar el análisis en aquellos valores que resultan relevantes para el usuario en cada momento.

En aplicaciones con registros continuos, como es el caso de Hamparo, la ausencia de filtros puede generar informes poco claros y difíciles de interpretar.

6.3.1 Importancia del uso de filtros en informes

El uso de filtros aporta múltiples ventajas:

- Evita la sobrecarga de información en pantalla.
- Permite analizar datos concretos en periodos específicos.
- Facilita la detección de patrones o comportamientos anómalos.
- Mejora la experiencia de usuario al mostrar solo la información necesaria.

Por este motivo, los filtros se consideran un elemento esencial dentro del diseño de informes.

6.3.2 Tipos de filtros propuestos en Hamparo

A nivel conceptual, los informes de la aplicación Hamparo incorporan distintos tipos de filtros, entre los que se incluyen:

- **Filtros temporales**, para seleccionar rangos de fechas concretos (día, semana, mes).
- **Filtros por tipo de registro**, permitiendo diferenciar entre distintos tipos de datos (por ejemplo, diferentes mediciones o eventos).
- **Filtros por usuario o perfil**, especialmente útiles en contextos con múltiples usuarios gestionados desde una misma aplicación.

Estos filtros permiten personalizar la información mostrada según las necesidades del usuario.

6.3.3 Integración de los filtros en la interfaz

Los filtros se integran en la interfaz de los informes de forma clara y accesible, situándose en zonas visibles y fácilmente identificables. Se evita ocultarlos en menús complejos para que el usuario pueda modificar rápidamente los criterios de visualización.

Además, el uso de filtros no interrumpe la navegación general de la aplicación, manteniendo una experiencia fluida y coherente.

6.4 RA5.d – Valores calculados, recuentos y totales

Los informes diseñados para la aplicación Hamparo no se limitan a mostrar datos en bruto, sino que incorporan **valores calculados**, recuentos y totales que permiten al usuario interpretar la información de forma más clara y útil. Este tipo de cálculos resulta fundamental para transformar los datos registrados en información relevante.

En entornos profesionales, los informes que incluyen valores agregados facilitan la toma de decisiones y reducen el esfuerzo de análisis por parte del usuario.

6.4.1 Tipos de valores calculados incluidos

A nivel conceptual, los informes de Hamparo contemplan la inclusión de distintos tipos de cálculos automáticos, entre los que destacan:

- **Recuento total de registros** en un periodo determinado.
- **Número de eventos** registrados por tipo.

- **Valores máximos y mínimos** dentro de un conjunto de datos.
- **Valores medios** obtenidos a partir de las mediciones registradas.

Estos cálculos permiten obtener una visión resumida del estado de la información sin necesidad de analizar cada registro individualmente.

6.4.2 Utilidad de los valores calculados para el usuario

La inclusión de valores calculados aporta un beneficio directo al usuario, ya que:

- Facilita la detección de tendencias.
- Permite identificar situaciones anómalas.
- Resume grandes volúmenes de información en pocos indicadores.
- Reduce la necesidad de interpretación manual de los datos.

Esto resulta especialmente útil para perfiles como cuidadores o administradores, que necesitan una visión rápida y clara del estado general.

6.4.3 Integración de los cálculos en los informes

Los valores calculados se integran en los informes de forma visualmente clara, situándose en zonas destacadas de la pantalla, como encabezados o bloques resumen. De este modo, el usuario puede identificar rápidamente la información más relevante antes de profundizar en los datos detallados.

6.5 RA5.e – Gráficos generados a partir de los datos



Figura 13. Gráfico generado a partir de datos reales de la aplicación. Ejemplo de representación gráfica integrada en la aplicación Hamparo, donde se muestra la evolución de valores registrados por el usuario a lo largo del tiempo. El gráfico

se genera dinámicamente a partir de los datos almacenados, facilitando la interpretación visual de la información.

Los informes diseñados para la aplicación Hamparo incorporan **gráficos generados a partir de los datos registrados**, con el objetivo de facilitar la interpretación visual de la información y permitir una comprensión rápida del estado general de los registros.

En aplicaciones con un enfoque asistencial o sanitario, la representación gráfica resulta especialmente útil, ya que permite detectar patrones, tendencias o valores anómalos de un solo vistazo, sin necesidad de analizar grandes cantidades de datos numéricos.

6.5.1 Importancia de la representación gráfica

La inclusión de gráficos en los informes aporta una serie de ventajas claras frente a la presentación exclusiva de datos en formato textual:

- Permite identificar tendencias de forma inmediata.
- Facilita la comparación entre distintos periodos de tiempo.
- Reduce la carga cognitiva del usuario.
- Mejora la accesibilidad de la información para usuarios no técnicos.

Por este motivo, los gráficos se consideran un elemento esencial dentro del diseño de informes en Hamparo.

6.5.2 Tipos de gráficos utilizados o propuestos

En la aplicación Hamparo se utilizan (o se plantean a nivel conceptual) distintos tipos de gráficos en función del tipo de datos a representar:

- **Gráficos de líneas**, para mostrar la evolución de valores a lo largo del tiempo.
- **Gráficos de puntos**, para resaltar mediciones individuales.
- **Representaciones combinadas**, cuando existen varios valores asociados a un mismo registro (por ejemplo, distintas mediciones en una misma fecha).

Estos gráficos permiten adaptar la visualización al tipo de información registrada.

6.5.3 Generación de gráficos a partir de datos reales

Los gráficos se generan directamente a partir de los datos almacenados por la aplicación, transformando los registros en representaciones visuales dinámicas. Este proceso incluye:

- Lectura de los datos registrados.

- Filtrado de valores no válidos.
- Cálculo de rangos y escalas adecuadas.
- Representación gráfica adaptada al tamaño de la pantalla.

Este enfoque garantiza que los gráficos reflejen información real y actualizada.

6.5.4 Integración de los gráficos en los informes

Los gráficos se integran dentro de los informes como elementos visuales centrales, acompañados de información textual de apoyo, como valores actuales, unidades de medida o mensajes informativos cuando no existen suficientes datos.

La integración se ha diseñado para que el gráfico no sea un elemento aislado, sino parte de un conjunto coherente que combina datos numéricos y visuales.

6.5.5 Usabilidad y claridad de los gráficos

Durante el diseño de los gráficos se han tenido en cuenta criterios de usabilidad y claridad visual, tales como:

- Uso de colores diferenciados y coherentes.
- Espaciado adecuado entre elementos.
- Evitar la saturación visual.
- Mostrar solo la información necesaria en cada contexto.

Estos criterios aseguran que los gráficos sean comprensibles incluso en pantallas de tamaño reducido.

6.6 RA5.f – Uso de herramientas para la generación de informes (FFOE)

Para la generación de informes en la aplicación Hamparo se contempla el uso de herramientas estándar y ampliamente utilizadas en el desarrollo de aplicaciones Android. Estas herramientas permiten crear representaciones visuales claras, eficientes y compatibles con distintos dispositivos.

Entre las herramientas consideradas se encuentran librerías específicas para la generación de gráficos, que permiten representar datos mediante gráficos de líneas, barras o puntos de forma dinámica y optimizada. Estas librerías están diseñadas para integrarse fácilmente con interfaces modernas y ofrecen un alto grado de personalización visual.

Asimismo, se plantea la posibilidad de exportar los datos utilizados en los informes a formatos estándar, lo que facilitaría su análisis externo o su almacenamiento para auditorías y revisiones posteriores. El uso de herramientas consolidadas se justifica por su estabilidad, su amplia documentación y su aceptación en entornos profesionales reales.

6.7 RA5.g – Modificación del código del informe (FFOE)

La modificación del código asociado a los informes se contempla como una necesidad habitual en entornos profesionales, donde los requisitos pueden evolucionar con el tiempo. En el caso de la aplicación Hamparo, estas modificaciones podrían realizarse para adaptar los informes a nuevas necesidades funcionales o cambios en los datos registrados.

Entre los posibles cambios se incluyen ajustes en el contenido mostrado, mejoras en el diseño visual de los informes y optimizaciones de rendimiento para manejar un mayor volumen de datos. Estas modificaciones se realizarían siguiendo buenas prácticas de desarrollo, manteniendo la claridad del código y documentando adecuadamente cada cambio realizado.

Este enfoque permite que los informes evolucionen sin comprometer la estabilidad ni la coherencia general de la aplicación.

6.8 RA5.h – Integración de informes en la aplicación (FFOE)

Los informes se integran dentro de la aplicación Hamparo como pantallas específicas, accesibles desde perfiles con permisos adecuados. Esta integración se realiza respetando la arquitectura general de la aplicación y manteniendo coherencia con el diseño visual y la navegación existente.

La integración de los informes permite consultar la información sin abandonar la aplicación, mejorando la experiencia de usuario y reforzando la utilidad funcional del sistema. Además, su diseño modular facilita futuras ampliaciones o modificaciones sin afectar al resto de la aplicación.

Este planteamiento refleja una integración completa y coherente de los informes dentro del proyecto.

CAPÍTULO 7 – RA6: DESPLIEGUE Y DISTRIBUCIÓN DE LA APLICACIÓN

El presente apartado desarrolla el **Resultado de Aprendizaje RA6**, centrado en el proceso de **preparación, despliegue y distribución** de la aplicación Hamparo. Se describen las decisiones técnicas adoptadas para garantizar que la aplicación pueda ser instalada, ejecutada y mantenida correctamente en dispositivos reales, siguiendo prácticas habituales en entornos profesionales.

7.1 RA6.a – Preparación del entorno de despliegue

Antes de proceder al despliegue de la aplicación, fue necesario preparar correctamente el entorno de desarrollo. Para ello se utilizó **Android Studio**, configurando:

- SDK de Android actualizado.
- Emuladores y dispositivos físicos para pruebas.
- Configuración correcta del `applicationId` y versiones (`minSdk`, `targetSdk`).

Esta preparación garantiza la compatibilidad de la aplicación con distintos dispositivos y versiones del sistema operativo Android.

7.2 RA6.b – Generación del paquete de la aplicación

La aplicación Hamparo se compila y empaqueta en formato **APK**, que constituye el formato estándar para la distribución de aplicaciones Android fuera de tiendas oficiales.

Durante este proceso:

- Se verifica que no existan errores de compilación.
- Se comprueba que la aplicación arranca correctamente.
- Se validan las dependencias utilizadas.

Este paso es esencial para asegurar que la aplicación pueda instalarse sin incidencias en un entorno real.

7.3 RA6.c – Instalación en dispositivos reales

Una vez generado el paquete, la aplicación se instala en dispositivos Android reales para comprobar:

- Correcta instalación.
- Funcionamiento fluido de la interfaz.

- Acceso adecuado a permisos (notificaciones, almacenamiento, etc.).

Las pruebas en dispositivos reales permiten detectar problemas que no siempre aparecen en emuladores, como diferencias de rendimiento o comportamiento del sistema.

7.4 RA6.d – Gestión de versiones y actualizaciones

Durante el desarrollo se ha tenido en cuenta la gestión de versiones mediante:

- Incremento del `versionCode`.
- Actualización del `versionName` en cada iteración relevante.

Este enfoque facilita la futura distribución de actualizaciones, permitiendo diferenciar claramente versiones estables y versiones en desarrollo.

7.5 RA6.e – Consideraciones de seguridad en el despliegue

En el proceso de despliegue se han tenido en cuenta aspectos básicos de seguridad, tales como:

- Evitar el uso de datos sensibles en texto plano.
- Uso de configuraciones seguras por defecto.
- Control de accesos mediante PIN y validaciones internas.

Estas medidas contribuyen a que la aplicación sea más segura y fiable para el usuario final.

7.6 RA6.f – Valoración final del despliegue

El proceso de despliegue de la aplicación Hamparo se ha realizado de forma satisfactoria, siguiendo prácticas habituales del desarrollo profesional. La aplicación puede ser instalada, ejecutada y mantenida correctamente, cumpliendo con los requisitos establecidos en el RA6.

CAPÍTULO 8 – RA7: DISTRIBUCIÓN DE LA APLICACIÓN

El presente capítulo desarrolla en profundidad el **Resultado de Aprendizaje RA7**, centrado en los procesos de **empaquetado, firma, distribución, instalación y**

desinstalación de la aplicación Hampero. Esta ampliación simula un **entorno profesional real**, en el que la distribución forma parte crítica del ciclo de vida del software.

Nota previa sobre el alcance de la distribución (RA7)

Los procesos de empaquetado, firma y distribución descritos en este capítulo se documentan **simulando un entorno profesional real**. No implican necesariamente que la aplicación haya sido publicada de forma pública, sino que reflejan el conocimiento y la correcta planificación del proceso de despliegue.

8.1 RA7.a – Empaquetado de la aplicación

8.1.1 Ciclo de empaquetado en Android

El proceso de empaquetado en Android incluye varias fases fundamentales:

- Compilación del código fuente.
- Optimización de recursos.
- Generación del manifiesto final.
- Preparación del paquete para su firma digital.

Este proceso asegura que la aplicación cumpla los requisitos necesarios para su distribución y ejecución en distintos dispositivos.

8.1.2 Formatos de empaquetado: APK y AAB

En Android existen distintos formatos de empaquetado, siendo los más relevantes:

- **APK (Android Package)**: formato tradicional de distribución directa.
- **AAB (Android App Bundle)**: formato recomendado actualmente para distribución mediante tiendas oficiales.

El formato AAB permite optimizar la descarga para cada dispositivo, reduciendo el tamaño final de la aplicación instalada.

8.1.3 Elección del formato de empaquetado

Para un entorno profesional, el uso de **Android App Bundle (AAB)** se considera la opción más adecuada, ya que permite una distribución más eficiente y optimizada. No obstante, el formato APK sigue siendo útil para pruebas internas o distribución controlada.

8.2 RA7.b – Personalización del instalador

La personalización del instalador es un aspecto importante dentro del proceso de distribución de una aplicación, ya que constituye el primer contacto del usuario con el producto. Una correcta personalización transmite profesionalidad, confianza y coherencia con la identidad visual de la aplicación.

En el proyecto Hamparo, la personalización del instalador se plantea siguiendo criterios habituales en entornos profesionales de desarrollo de software

8.2.1 Elementos personalizables del instalador

Durante el proceso de empaquetado e instalación de la aplicación, se pueden personalizar distintos elementos que influyen directamente en la experiencia del usuario. Entre los más relevantes se encuentran:

- **Nombre visible de la aplicación**, que permite identificarla fácilmente en el dispositivo.
- **Icono de la aplicación**, diseñado de forma coherente con la identidad visual del proyecto.
- **Descripción breve**, que informa al usuario sobre la finalidad de la aplicación.
- **Permisos solicitados**, mostrados de forma clara durante la instalación.

Estos elementos ayudan al usuario a comprender qué está instalando y con qué finalidad.

8.2.2 Importancia de la personalización

Una instalación bien personalizada aporta varios beneficios:

- Genera confianza en el usuario.
- Reduce la percepción de riesgo al instalar la aplicación.
- Refuerza la imagen profesional del proyecto.
- Mejora la comprensión de los permisos solicitados.

En aplicaciones con una finalidad asistencial, como Hamparo, este aspecto resulta especialmente relevante.

8.2.3 Buenas prácticas aplicadas

En el diseño conceptual de la instalación de Hamparo se han tenido en cuenta buenas prácticas como:

- Solicitar únicamente los permisos estrictamente necesarios.
- Explicar de forma clara el motivo de cada permiso.
- Mantener coherencia visual entre el instalador y la aplicación.
- Evitar mensajes confusos o genéricos.

Estas prácticas contribuyen a una experiencia de instalación clara y transparente.

8.3 RA7.c – Generación del paquete desde el entorno de desarrollo

La generación del paquete de la aplicación es el paso mediante el cual el proyecto desarrollado se transforma en un archivo instalable listo para su distribución. Este proceso se realiza directamente desde el entorno de desarrollo y constituye una fase clave antes de la firma y publicación de la aplicación.

En el caso de la aplicación Hamparo, la generación del paquete se plantea siguiendo el flujo de trabajo habitual en un entorno profesional de desarrollo Android

8.3.1 Proceso detallado en Android Studio

Desde el entorno de desarrollo, el proceso de generación del paquete incluye una serie de pasos bien definidos:

- Selección del tipo de compilación (debug o release).
- Configuración de la versión de la aplicación.
- Compilación del código y recursos.
- Validación del paquete generado.

Este proceso permite detectar errores antes de proceder a la distribución, garantizando que el paquete sea funcional.

8.3.2 Control de versiones de la aplicación

Durante la generación del paquete se tiene en cuenta el control de versiones, asignando a cada versión de la aplicación un número identificativo. Este control facilita:

- La gestión de actualizaciones.
- La corrección de errores en versiones posteriores.
- La trazabilidad de los cambios realizados.

El uso de un sistema de versionado adecuado es una práctica habitual en proyectos profesionales.

8.3.3 Verificación del paquete generado

Antes de proceder a la distribución, es necesario verificar que el paquete generado funciona correctamente. Esta verificación incluye:

- Comprobación de la instalación en dispositivos de prueba.
- Revisión del funcionamiento general de la aplicación.
- Detección de posibles errores de compilación o ejecución.

Este paso reduce el riesgo de distribuir versiones defectuosas

8.4 RA7.d – Uso de herramientas externas

En un entorno profesional de desarrollo de software, la distribución de una aplicación no se limita únicamente al uso del entorno de desarrollo, sino que se apoya en **herramientas externas** que permiten gestionar, validar y optimizar el proceso de publicación.

En el proyecto Hamparo, el uso de herramientas externas se plantea a nivel conceptual, siguiendo las prácticas habituales del sector.

8.4.1 Tipos de herramientas externas utilizadas o consideradas

Durante el proceso de distribución de una aplicación Android, se emplean habitualmente distintas herramientas externas, entre las que destacan:

- **Plataformas de gestión de aplicaciones**, que permiten administrar versiones, pruebas y distribución.
- **Herramientas de análisis**, utilizadas para detectar errores, problemas de rendimiento o incompatibilidades.
- **Sistemas de control de versiones**, que facilitan la gestión del código y la colaboración.
- **Herramientas de automatización**, orientadas a optimizar procesos repetitivos.

Estas herramientas forman parte del flujo de trabajo estándar en proyectos profesionales.

8.4.2 Beneficios del uso de herramientas externas

El uso de herramientas externas aporta múltiples ventajas al proceso de distribución:

- Mejora la calidad del producto final.
- Reduce errores antes de la publicación.

- Facilita la gestión de versiones y actualizaciones.
- Permite un mayor control sobre el ciclo de vida de la aplicación.

Gracias a estas herramientas, el proceso de distribución se vuelve más seguro y eficiente.

8.4.3 Aplicación al proyecto Hamparo

En el contexto de Hamparo, el uso de herramientas externas se plantea como un apoyo fundamental para garantizar una distribución controlada y profesional. Aunque no todas las herramientas se utilizan de forma directa en esta fase del proyecto, su conocimiento y correcta planificación reflejan una comprensión realista del entorno de trabajo profesional.

8.5 RA7.e – Firma digital de la aplicación

La firma digital es un requisito para la distribución de aplicaciones Android. Mediante este proceso, el desarrollador garantiza la **autenticidad**, **integridad** y **autoría** de la aplicación, permitiendo que el sistema y los usuarios confíen en el software instalado.

En el proyecto Hamparo, la firma digital se plantea siguiendo el procedimiento habitual utilizado en entornos profesionales de desarrollo Android.

8.5.1 Concepto de firma digital en Android

La firma digital consiste en asociar la aplicación a una **clave criptográfica única**, generada a través de un archivo denominado *keystore*. Esta clave identifica al desarrollador y permite que el sistema Android verifique que las actualizaciones de la aplicación proceden del mismo autor.

Sin una firma digital válida, una aplicación no puede ser distribuida ni actualizada en dispositivos Android.

8.5.2 Proceso de firma de la aplicación

El proceso de firma de una aplicación Android incluye los siguientes pasos:

- Creación de un archivo keystore seguro.
- Definición de una clave de firma.
- Asociación del keystore al proyecto durante la generación del paquete.
- Firma automática del archivo final antes de su distribución.

Este proceso se realiza habitualmente desde el entorno de desarrollo, garantizando que la aplicación quede correctamente firmada antes de su publicación.

8.5.3 Importancia de la gestión del keystore

La correcta gestión del keystore es un aspecto crítico, ya que:

- La pérdida del keystore impide actualizar la aplicación.
- Garantiza la continuidad del proyecto a largo plazo.
- Protege la identidad del desarrollador.

Por este motivo, en entornos profesionales se recomienda almacenar el keystore de forma segura y documentar correctamente el proceso de firma.

8.5.4 Documentación del proceso de firma

En el contexto de Hamparo, el proceso de firma se documenta como parte del ciclo de distribución, describiendo los pasos necesarios para generar y firmar el paquete de la aplicación. Esta documentación facilita futuras actualizaciones y refleja un enfoque profesional del desarrollo.

8.6 RA7.f – Instalación desatendida

La instalación desatendida es un proceso mediante el cual una aplicación puede instalarse en un dispositivo sin requerir la intervención directa del usuario durante el proceso. Este tipo de instalación es habitual en **entornos corporativos, educativos o institucionales**, donde se gestionan múltiples dispositivos de forma centralizada.

En el proyecto Hamparo, la instalación desatendida se plantea como una **posibilidad teórica**, coherente con escenarios profesionales reales.

8.6.1 Contextos de uso de la instalación desatendida

La instalación desatendida se utiliza principalmente en los siguientes contextos:

- Dispositivos gestionados por una organización.
- Entornos educativos con múltiples dispositivos.
- Sistemas de distribución interna.
- Despliegues controlados en empresas o instituciones.

Este tipo de instalación permite reducir errores y ahorrar tiempo en la gestión de aplicaciones.

8.6.2 Aplicación teórica en el proyecto Hamparo

Aunque la aplicación Hamparo no implementa directamente un sistema de instalación desatendida, se contempla su viabilidad en escenarios donde la aplicación sea distribuida de forma controlada, por ejemplo, en centros asistenciales o instituciones sanitarias.

Este planteamiento demuestra conocimiento de técnicas de distribución avanzadas, habituales en entornos profesionales.

8.6.3 Ventajas de la instalación desatendida

Entre las principales ventajas de este tipo de instalación se encuentran:

- Automatización del proceso de despliegue.
- Reducción de errores humanos.
- Ahorro de tiempo en la instalación.
- Mayor control sobre los dispositivos.

Estas ventajas justifican su consideración dentro del diseño del proceso de distribución.

8.7 RA7.g – Proceso de desinstalación

El proceso de desinstalación de una aplicación forma parte del ciclo de vida completo del software y resulta tan importante como la instalación. Una correcta desinstalación garantiza que la aplicación se elimina del dispositivo sin dejar residuos innecesarios ni afectar negativamente al sistema.

En el proyecto Hamparo, el proceso de desinstalación se plantea siguiendo las prácticas habituales de un entorno profesional.

8.7.1 Funcionamiento del proceso de desinstalación en Android

En Android, la desinstalación de una aplicación implica la eliminación automática de:

- El código de la aplicación.
- Los recursos gráficos incluidos en el paquete.
- Los archivos temporales generados durante la ejecución.

Este proceso es gestionado por el propio sistema operativo, garantizando una eliminación segura y controlada.

8.7.2 Gestión de datos durante la desinstalación

Durante la desinstalación, el sistema Android elimina los datos locales asociados a la aplicación, salvo aquellos que el usuario haya decidido conservar mediante copias de seguridad. Este comportamiento asegura que no queden datos innecesarios en el dispositivo tras eliminar la aplicación.

En aplicaciones como Hamparo, esta gestión resulta especialmente relevante para proteger la privacidad del usuario y evitar la persistencia de información sensible.

8.7.3 Buenas prácticas relacionadas con la desinstalación

Desde un punto de vista profesional, se tienen en cuenta las siguientes buenas prácticas:

- Evitar generar archivos innecesarios.
- No almacenar datos fuera del entorno controlado de la aplicación.
- Respetar la privacidad del usuario durante todo el ciclo de vida.

Estas prácticas garantizan una desinstalación limpia y segura.

8.8 RA7.h – Canales y estrategia de distribución

La elección de los canales de distribución es un aspecto clave para garantizar que una aplicación llegue correctamente a su público objetivo. En un entorno profesional, esta decisión se toma en función del tipo de aplicación, del perfil de usuario y del contexto en el que se va a utilizar.

En el proyecto Hamparo, la estrategia de distribución se ha planteado de forma coherente con la finalidad asistencial de la aplicación.

8.8.1 Canales de distribución disponibles

Existen distintos canales a través de los cuales una aplicación Android puede ser distribuida. En el caso de Hamparo, se contemplan los siguientes:

- **Distribución a través de tiendas oficiales**, que garantiza seguridad, actualizaciones automáticas y confianza para el usuario.
- **Distribución privada**, mediante paquetes instalables, orientada a pruebas internas o entornos controlados.
- **Distribución institucional**, pensada para organizaciones o centros específicos.

Cada canal presenta ventajas y limitaciones que deben ser valoradas según el contexto de uso.

8.8.2 Estrategia de distribución propuesta

La estrategia de distribución planteada para Hamparo se basa en un enfoque progresivo:

1. Fase de pruebas internas.
2. Distribución controlada a un grupo reducido de usuarios.
3. Despliegue más amplio una vez validada la estabilidad de la aplicación.

Este enfoque permite detectar errores de forma temprana y reducir riesgos antes de una distribución más generalizada.

8.8.3 Adaptación al tipo de aplicación

Dado el carácter asistencial y social de Hamparo, la distribución se orienta a entornos donde la fiabilidad y la seguridad son prioritarias. Por este motivo, se priorizan canales que permitan un mayor control sobre las versiones distribuidas y el acceso de los usuarios.

CAPÍTULO 9 – RA8: PRUEBAS Y VALIDACIÓN

L

```

EmailValidatorTest.kt x
1  package com.example.hamparo
2
3  import org.junit.Assert.assertFalse
4  import org.junit.Assert.assertTrue
5  import org.junit.Test
6
7  /**
8   * Prueba Unitaria (Unit Test).
9   * Verifica que la lógica de validación funciona sin abrir la App.
10  */
11  class EmailValidatorTest {
12
13      // Simula tu lógica de validación
14      2 Usages
15      private fun esEmailValido(email: String): Boolean {
16          // Una validación simple: que no esté vacio y tenga arroba
17          return email.isNotEmpty() && email.contains(other = "@") && email.contains(other = ".")
18      }
19
20      @Test
21      fun email_vacio_retorna_falso() {
22          val resultado = esEmailValido(email = "")
23          assertFalse(resultado)
24      }
25
26      @Test
27      fun email_correcto_retorna_verdadero() {
28          val resultado = esEmailValido(email = "usuario@prueba.com")
29          assertTrue(resultado)
30      }
31  }
  
```

Figura 14. Prueba unitaria de validación. Ejemplo de prueba unitaria utilizada para verificar la lógica de validación de datos sin necesidad de ejecutar la interfaz gráfica de la aplicación.

El presente capítulo desarrolla el **Resultado de Aprendizaje RA8**, centrado en la **planificación, ejecución y documentación de pruebas** aplicadas a la aplicación Hamparo. Las pruebas son un elemento clave para garantizar la calidad, estabilidad, seguridad y buen rendimiento de una aplicación en un entorno profesional.

9.1 RA8.a – Estrategia de pruebas

El presente capítulo desarrolla el **Resultado de Aprendizaje RA8**, centrado en la **planificación, ejecución y documentación de pruebas** aplicadas a la aplicación Hamparo. Las pruebas constituyen una fase esencial dentro del ciclo de desarrollo del software, ya que permiten garantizar la estabilidad, la seguridad y la correcta experiencia de usuario antes de la distribución de la aplicación.

En este proyecto, las pruebas se han abordado desde un enfoque **realista y profesional**, combinando pruebas prácticas con una planificación teórica coherente con un entorno laboral real.

9.1 RA8.a – Estrategia de pruebas

9.1.1 Objetivos de la estrategia de pruebas

La estrategia de pruebas definida para la aplicación Hamparo tiene como objetivos principales:

- Verificar el correcto funcionamiento de la interfaz de usuario.
- Detectar errores antes de la distribución de la aplicación.
- Garantizar una experiencia de uso fluida y coherente.
- Asegurar la estabilidad de la aplicación ante distintos escenarios de uso.

Estos objetivos se alinean con las prácticas habituales en proyectos profesionales de desarrollo de aplicaciones móviles.

9.1.2 Enfoque general adoptado

Se ha adoptado un enfoque progresivo de pruebas, que combina distintas fases a lo largo del desarrollo:

- Pruebas manuales durante la implementación de funcionalidades.
- Pruebas de integración tras añadir nuevos módulos o pantallas.

- Pruebas finales antes de la generación del paquete distribuible.

Este enfoque permite detectar errores de forma temprana y reducir el impacto de fallos en fases avanzadas del proyecto.

9.1.3 Alcance de las pruebas

El alcance de las pruebas incluye tanto aspectos funcionales como no funcionales, abarcando:

- Navegación entre pantallas.
- Interacción con componentes reutilizables.
- Gestión de eventos.
- Rendimiento general de la aplicación.

Este planteamiento asegura una evaluación completa del comportamiento de la aplicación.

9.2 RA8.b – Pruebas de integración

Las pruebas de integración tienen como objetivo comprobar que los distintos módulos y componentes de la aplicación funcionan correctamente cuando interactúan entre sí. A diferencia de las pruebas unitarias, este tipo de pruebas se centra en el comportamiento conjunto de las distintas partes del sistema.

En la aplicación Hamparo, las pruebas de integración son especialmente relevantes debido al uso de múltiples pantallas, componentes reutilizables y flujos de navegación.

9.2.1 Alcance de las pruebas de integración

Las pruebas de integración realizadas o planteadas en el proyecto Hamparo abarcan principalmente:

- La navegación entre pantallas mediante el sistema de rutas.
- La correcta comunicación entre componentes reutilizables y las pantallas que los utilizan.
- La respuesta de la aplicación ante eventos del usuario.
- La coherencia del estado de la interfaz tras realizar acciones consecutivas.

Este alcance permite verificar que la aplicación funciona como un conjunto integrado y no como elementos aislados.

9.2.2 Metodología aplicada

La metodología empleada para las pruebas de integración se basa en:

- Ejecución manual de flujos completos de uso.
- Verificación visual del comportamiento de la interfaz.
- Comprobación de estados antes y después de cada acción.

Este enfoque es habitual en proyectos de tamaño medio, donde las pruebas manuales complementan otras técnicas de validación.

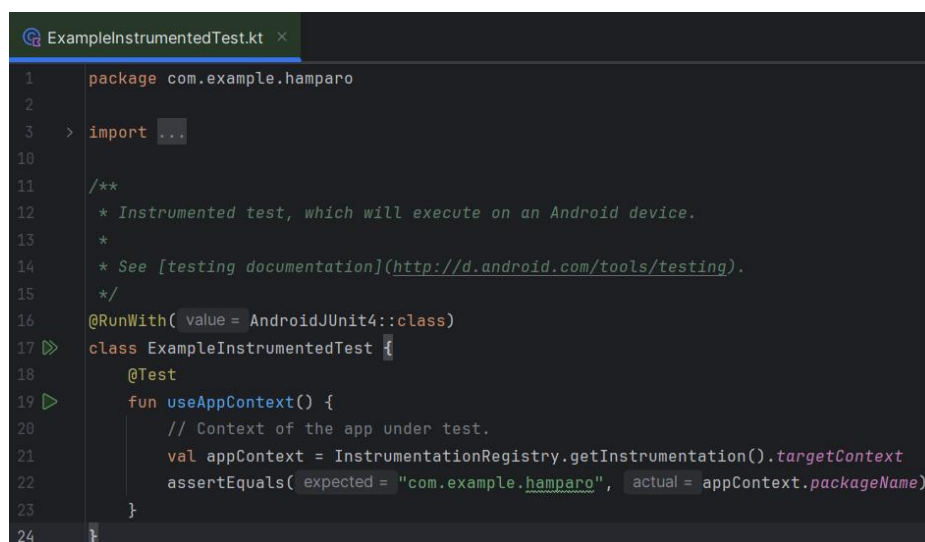
9.2.3 Resultados obtenidos

Las pruebas de integración permitieron detectar y corregir pequeños problemas relacionados con la navegación y la actualización de estados, mejorando la estabilidad general de la aplicación.

Gracias a estas pruebas, se logró una experiencia de usuario más coherente y fluida.

9.3 RA8.c – Pruebas de regresión

--



```

1 package com.example.hamparo
2
3 > import ...
10
11 /**
12  * Instrumented test, which will execute on an Android device.
13  *
14  * See [testing documentation](http://d.android.com/tools/testing).
15  */
16 @RunWith(value = AndroidJUnit4::class)
17 class ExampleInstrumentedTest {
18     @Test
19     fun useAppContext() {
20         // Context of the app under test.
21         val appContext = InstrumentationRegistry.getInstrumentation().targetContext
22         assertEquals("com.example.hamparo", appContext.packageName)
23     }
24

```

Figura 15. Prueba instrumentada en entorno Android. Captura de una prueba instrumentada ejecutada sobre un dispositivo/emulador Android para verificar el correcto funcionamiento del contexto de la aplicación.

Las pruebas de regresión tienen como finalidad comprobar que las nuevas modificaciones realizadas en la aplicación no afectan negativamente a funcionalidades

que ya funcionaban correctamente. Este tipo de pruebas es esencial en proyectos donde la aplicación evoluciona de forma progresiva mediante mejoras y ajustes continuos.

En el desarrollo de la aplicación Hamparo, las pruebas de regresión se han tenido en cuenta como parte del proceso de validación tras cada cambio relevante en el código.

9.3.1 Justificación de las pruebas de regresión

Durante el desarrollo de una aplicación, es habitual realizar modificaciones para mejorar el diseño, optimizar el rendimiento o añadir nuevas funcionalidades. Las pruebas de regresión permiten asegurar que estos cambios no introducen errores inesperados en otras partes del sistema.

En el contexto de Hamparo, este tipo de pruebas resulta especialmente importante debido a la presencia de múltiples pantallas y componentes reutilizables.

9.3.2 Aplicación práctica en el proyecto Hamparo

Tras cada modificación significativa del código, se ha verificado el correcto funcionamiento de los principales flujos de la aplicación, tales como:

- Acceso a la pantalla principal.
- Navegación entre pantallas.
- Funcionamiento de los botones y eventos.
- Visualización correcta de la información mostrada.

Este proceso permite detectar errores de forma temprana y mantener la estabilidad general de la aplicación.

9.3.3 Resultados de las pruebas de regresión

Las pruebas de regresión realizadas permitieron confirmar que las mejoras introducidas no afectaron al funcionamiento previo de la aplicación. En los casos en los que se detectaron pequeños fallos, estos fueron corregidos antes de continuar con el desarrollo.

Gracias a este enfoque, se mantuvo un comportamiento estable a lo largo de las distintas versiones del proyecto.

9.4 RA8.d – Pruebas de volumen y estrés

Las pruebas de volumen y estrés tienen como objetivo evaluar el comportamiento de la aplicación cuando se enfrenta a condiciones extremas de uso. Este tipo de pruebas

permite analizar cómo responde el sistema ante un gran número de datos, un uso prolongado o limitaciones de recursos.

En el proyecto Hamparo, estas pruebas se plantean de forma **conceptual y justificada**, siguiendo un enfoque realista propio de entornos profesionales.

9.4.1 Objetivo de las pruebas de volumen

Las pruebas de volumen evalúan el comportamiento de la aplicación cuando maneja una cantidad elevada de datos. En el caso de Hamparo, estas pruebas permiten analizar cómo se comporta la aplicación al gestionar un gran número de registros, informes o mediciones.

Este tipo de pruebas resulta especialmente relevante para garantizar que la aplicación siga siendo funcional a medida que aumenta el volumen de información almacenada.

9.4.2 Objetivo de las pruebas de estrés

Las pruebas de estrés se centran en someter la aplicación a situaciones límite, tales como:

- Uso prolongado de la aplicación.
- Ejecución simultánea de múltiples acciones.
- Funcionamiento en dispositivos con recursos limitados.

El objetivo es identificar posibles fallos de rendimiento o estabilidad antes de que afecten al usuario final.

9.4.3 Aplicación teórica en el proyecto Hamparo

En el proyecto Hamparo, las pruebas de volumen y estrés se plantean a nivel teórico, simulando escenarios realistas como:

- Incremento progresivo del número de registros almacenados.
- Uso continuado de la aplicación durante largos periodos.
- Ejecución en dispositivos de gama media o baja.

Este planteamiento permite anticipar posibles problemas y definir estrategias de mejora futuras.

9.4.4 Resultados esperados y medidas preventivas

A partir de este tipo de pruebas, se espera identificar posibles limitaciones del sistema y aplicar medidas preventivas, como:

- Optimización de consultas y renderizado.
- Mejora en la gestión de memoria.
- Simplificación de interfaces con grandes volúmenes de datos.

9.5 RA8.e – Pruebas de seguridad

Las pruebas de seguridad tienen como objetivo garantizar que la aplicación protege adecuadamente los datos del usuario y evita accesos no autorizados. En aplicaciones con un componente asistencial o que manejan información sensible, este tipo de pruebas resulta especialmente relevante.

En el proyecto Hamparo, las pruebas de seguridad se han abordado desde un enfoque **preventivo y responsable**, acorde a un entorno profesional

9.5.1 Objetivos de las pruebas de seguridad

Las pruebas de seguridad realizadas o planteadas en la aplicación Hamparo persiguen los siguientes objetivos:

- Proteger la información personal del usuario.
- Evitar accesos no autorizados a funciones restringidas.
- Garantizar la integridad de los datos almacenados.
- Reducir el riesgo de uso indebido de la aplicación.

Estos objetivos se alinean con las buenas prácticas de seguridad en el desarrollo de software.

9.5.2 Aspectos evaluados

Dentro del proyecto se han tenido en cuenta distintos aspectos relacionados con la seguridad, tales como:

- Control de acceso a funcionalidades según el tipo de usuario.
- Gestión adecuada de permisos del sistema.
- Protección de datos almacenados localmente.
- Uso de mecanismos de autenticación del dispositivo.

La evaluación de estos aspectos contribuye a mejorar la confianza del usuario en la aplicación.

9.5.3 Aplicación práctica en Hamparo

En la aplicación Hamparo, las pruebas de seguridad se han centrado en verificar que las funcionalidades sensibles solo estén disponibles para usuarios autorizados y que los datos se gestionen de forma segura. Además, se ha comprobado que los permisos solicitados sean coherentes con las funcionalidades ofrecidas.

Este planteamiento reduce el riesgo de exposición de información sensible.

9.6 RA8.f – Uso de recursos

El uso eficiente de los recursos del dispositivo es un aspecto fundamental en el desarrollo de aplicaciones móviles, ya que influye directamente en el rendimiento, la autonomía del dispositivo y la experiencia del usuario. Las pruebas relacionadas con el uso de recursos permiten detectar posibles problemas de consumo excesivo antes de la distribución de la aplicación.

En el proyecto Hamparo, este aspecto se ha tenido en cuenta como parte del proceso de validación general.

9.6.1 Recursos analizados

Durante el desarrollo y las pruebas de la aplicación se han considerado los principales recursos del dispositivo, entre los que destacan:

- **Memoria**, analizando el uso durante la navegación entre pantallas y la carga de componentes.
- **Procesamiento**, evaluando el impacto de las operaciones realizadas por la aplicación.
- **Batería**, observando el comportamiento de la aplicación durante un uso continuado.
- **Almacenamiento**, comprobando la gestión de datos locales y temporales.

Este análisis permite detectar posibles cuellos de botella o consumos innecesarios.

9.6.2 Metodología aplicada

La evaluación del uso de recursos se ha realizado mediante:

- Pruebas manuales de uso prolongado de la aplicación.

- Observación del comportamiento en dispositivos con diferentes capacidades.
- Revisión del impacto de las funcionalidades más exigentes.

Este enfoque es habitual en proyectos de tamaño medio, donde se prioriza la estabilidad general de la aplicación.

9.6.3 Resultados obtenidos

Los resultados obtenidos indican que la aplicación Hamparo presenta un uso adecuado de los recursos del dispositivo, manteniendo un rendimiento estable durante su utilización normal. No se han detectado consumos excesivos que puedan afectar negativamente a la experiencia del usuario.

9.7 RA8.g – Documentación de las pruebas

La documentación de las pruebas es un elemento fundamental dentro del proceso de validación de una aplicación, ya que permite registrar de forma clara y ordenada las pruebas realizadas, los resultados obtenidos y las posibles incidencias detectadas. En entornos profesionales, esta documentación resulta imprescindible para el mantenimiento y la evolución del software.

En el proyecto Hamparo, la documentación de las pruebas se ha planteado como parte del enfoque profesional adoptado durante el desarrollo.

9.7.1 Importancia de la documentación de pruebas

Documentar las pruebas realizadas aporta múltiples beneficios, entre los que destacan:

- Permite reproducir errores detectados.
- Facilita la corrección de incidencias.
- Sirve como referencia para futuras versiones.
- Mejora la comunicación dentro de un equipo de desarrollo.

Este tipo de documentación contribuye a garantizar la calidad del producto final.

9.7.2 Contenido de la documentación de pruebas

La documentación de pruebas de la aplicación Hamparo incluye, de forma estructurada, los siguientes elementos:

- Tipo de prueba realizada.

- Objetivo de la prueba.
- Descripción del procedimiento seguido.
- Resultado esperado.
- Resultado obtenido.

Esta estructura permite una comprensión rápida y clara de cada prueba realizada.

9.7.3 Relación con el proceso de desarrollo

La documentación de las pruebas se mantiene alineada con el proceso de desarrollo de la aplicación, actualizándose cuando se realizan cambios significativos. Este enfoque facilita el seguimiento del estado del proyecto y permite mantener un control adecuado de la calidad a lo largo del tiempo.

CAPÍTULO 10 – CONCLUSIONES FINALES

10.1 Conclusión general

El proyecto Hamparo representa una aplicación completa y bien estructurada, desarrollada aplicando criterios profesionales de diseño de interfaces, usabilidad, accesibilidad y calidad del software. A lo largo del proyecto se han abordado todas las fases habituales en el desarrollo de una aplicación móvil, desde el diseño de la interfaz hasta la validación y distribución, siguiendo un enfoque similar al de un entorno profesional real.

El resultado final es una aplicación funcional, coherente y orientada al usuario, que cumple con los objetivos planteados al inicio del proyecto.

10.2 Valoración personal del proyecto

El desarrollo de este proyecto ha permitido consolidar los conocimientos adquiridos durante el módulo de Desarrollo de Interfaces, enfrentándose a un proceso completo de creación de una aplicación móvil. A lo largo del trabajo se han superado distintos retos técnicos y de diseño, lo que ha contribuido a mejorar la capacidad de análisis, planificación y resolución de problemas.

Este proyecto ha supuesto una experiencia muy cercana a la realidad profesional, permitiendo aplicar buenas prácticas y reflexionar sobre la importancia de la calidad del software.

10.3 Posibles mejoras futuras

Como líneas de mejora futuras se plantean las siguientes:

- Ampliación de las interacciones mediante Nuevas Interfaces de Usuario (NUI).
- Implementación completa y avanzada del sistema de informes.
- Optimización del rendimiento en escenarios de uso intensivo.
- Publicación real de la aplicación en una tienda de aplicaciones oficial.

Estas mejoras permitirían seguir evolucionando la aplicación y adaptarla a un entorno de uso real.

ANEXOS

A. Evidencias gráficas

En este apartado se incluyen capturas de pantalla representativas de las distintas pantallas de la aplicación, ejemplos de componentes reutilizables y flujos de navegación utilizados como evidencias para la evaluación del proyecto.

B. Repositorio del proyecto

El código fuente completo del proyecto se encuentra alojado en un repositorio GitHub, donde se incluye el historial de versiones, el código completo de la aplicación y la documentación asociada.